AIR FARE PREDICTION SYSTEM

A PROJECT REPORT

In partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

By

P. NITHIN KUMAR (19001A0548)

S. CHANAKYA SAI (19001A0549)

VEMULA SAI (19001A0507)

Under the guidance of

Dr. B. ESWARA REDDY M.Tech, Ph.D.

Professor of Computer Science and Engineering

JNTUACEA



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY ANANTAPUR

COLLEGE OF ENGINEERING (Autonomous)

ANANTAPURAMU-515002

ANDHRA PRADESH

2022-23

JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY ANANTAPUR
COLLEGE OF ENGINEERING (Autonomous)
Ananthapuramu, Andhra Pradesh-515002
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE

This is to certify that the project entitled **"AIR FARE PREDICTION SYSTEM"** is a bonafide work done by **P. Nithin Kumar - 19001A0548, S. Chanakya Sai - 19001A0549, Vemula Sai - 19001A0507,** in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering, from Jawaharlal Nehru Technological University Anantapur College of Engineering, Ananthapuramu during the year 2022 -2023.

Signature of Project Guide

Dr. B. Eswara Reddy, **M.Tech, Ph.D.,**
Professor,
Department of CSE,
JNTUA College of Engineering,
Ananthapuramu– 515002

Signature of H.O.D

Dr. K.F. Bharati , **M.Tech, Ph.D.,**
Associate Professor & H.O.D,
Department of CSE,
JNTUA College of Engineering,
Ananthapuramu– 515002

Internal Examiner

External Examiner

DECLARATION

We hereby declare that the project report entitled "**AIR FARE PREDICTION SYSTEM**" was done by us under the esteemed guidance of **Dr. B. ESWARA REDDY, Professor** and is submitted in partial fulfilment of the requirements for the award of the Bachelor's of Technology in **Computer Science and Engineering.**

Date:

Place:

P. Nithin Kumar (19001A0548)

S. Chanakya Sai (19001A0549)

Vemula Sai (19001A0507)

ACKNOWLEDGEMENTS

The satisfaction and euphoria that accompany the successful completion of any task would be incomplete without the mention of people who made it possible. As there is guidance and encouragement crowned our efforts with success. It is a pleasant aspect that we now have the opportunity to express my gratitude to all of them.

We are highly indebted to **Dr. B. ESWARA REDDY**, Professor of Computer Science and Engineering Department, JNTUACE Ananthapuramu. His wide knowledge and logical way of thinking have made a deep impression on us. His understanding, thinking and personal guidance have provided the basis for the project. His guidance and constant supervision helped us in the completion of this project.

We would like to express special thanks to **Dr. K.F. BHARATI**, Head of the department, Computer Science and Engineering, JNTUACE Ananthapuramu, Her wide support, knowledge and enthusiastic encouragement have impressed us to better involvement in our project and technical design.

As we are grateful to Prof. **P. SUJATHA**, Principal of JNTUACE Ananthapuramu, for her indispensable encouragement and cooperation to complete the project.

Successful completion of any project cannot be done without the support and encouragement. We sincerely thank all teaching and non-teaching staff of Computer Science and Engineering Department for providing all the necessary facilities during the course of study.

We would like to thank our parents and friends, who have the greatest contributions to all our achievements, for the great care and blessings in making us successful in all our endeavors.

With Gratitude,

P. NITHIN KUMAR (19001A0548)

S. CHANAKYA SAI (19001A0549)

VEMULA SAI (19001A0507)

TABLE OF CONTENTS

ABSTRACT 7
CHAPTER 1 : INTRTODUCTION 8
1.1 Problem Definition: 8
1.2 Motivation: 8
1.3 Problem Statement: 9
1.4 Objectives: 9
1.6 Random Forest Algorithm: 9
1.7 Google Colab: 11
CHAPTER 2 : SYSTEM ANALYSIS 12
2.1 Existing System: 12
2.2 Proposed System: 13
2.3 Requirement Analysis: 13
2.3.1 Functional Requirements: 13
2.3.2 Non-Functional Requirements: 14
2.3.3 Hardware Requirements 15
2.3.4 Software Requirements 15
CHAPTER 3 : SYSTEM DESIGN 16
3.1 System Architecture 16
3.2 Data Flow Diagram 17
3.3 Use Case Diagram 18
3.4 Sequence Diagram 19
3.5 Class Diagram 20
3.6 Activity Diagram 21
CHAPTER 4 : SYSTEM IMPLEMENTATION 22
4.1 Setup Prerequisites 22
4.2 Downloading The Dataset 22
4.3 Importing The Dataset 23
4.3 Data Preprocessing 24
4.4 EDA Of Train Dataset 25
4.5 Handling The Categorical Data 27
4.6 Test Dataset 31
4.7 EDA Of Test Dataset 31

4.8 Feature Extraction 34
4.9 Using Extra Tree Regressor 35
4.10 Fitting Model With Random Forest Algorithm 35
4.11 Fitting Model With Hyperparameter Tuning 37
4.12 Saving The Model For Pickle 39
4.13 Final Accuracy Of The Model 40
4.14 Features 40
CHAPTER 5 : TESTING 41
5.1 Model Testing 41
5.2 Unit Testing 42
5.3 Performance Testing 43
CHAPTER 6 : RESULTS AND DISCUSSION 44
6.1 Data Handling: 44
6.1.1 One Hot Encoding: 44
6.1.2 Label Encoding: 45
6.1.3 Categorical Data: 45
6.2 Model Training: 46
6.2.1 Fitting Model with Random Forest Algorithm: 47
6.2.2 Hyper Parameter Tuning: 48
6.3 Correlation for Independent and Dependent Features: 50
6.4 Feature Selection: 51
6.5 Model Deployment: 51
CHAPTER 7 : CONCLUSION AND FUTURE WORK 52
7.1 Conclusion 52
7.2 Limitations 54
7.3 Future Work 54
REFERENCES 55

AIR FARE PREDICTION SYSTEM

ABSTRACT

Airlines play an important role in today’s world, serving as a key driver of global travel, trade, and economic growth. Since travelling by plane has become an integral part of today's lifestyle as more and more people opt for faster travel options, it is the fastest-growing industry at present. By having various kinds of classes, like economy, first class and high flight ticket prices, they generate huge profits.

Flight ticket prices can be somewhat hard to guess as they fluctuate constantly, so purchasing at different times could mean large differences in price. On the customer side, models are used to predict the optimal time to buy a ticket at a minimum price. Customers are seeking all-time low prices, while airlines are attempting to keep their overall revenue as high as possible and maximize their profit.

Airlines use different kinds of techniques to increase their profits, like demand prediction and price inequity. In the proposed system, a predictive model will be created by applying machine learning algorithms to the collected historical data on flights. This system will give people an idea of the trends that prices follow and also provide a predicted price value that they can refer to before booking their flight tickets in order to save money.

CHAPTER 1
INTRODUCTION

The project aims to develop a machine learning-based application to predict flight prices and help users book tickets at the right time by predicting prices of flights by their past trends. The motivation is to save customers money by avoiding overpaying for tickets. The scope includes studying factors influencing flight prices and building a prediction system.

1.1 Problem Definition:

This project aims to develop an application which will predict the flight prices for various flights using machine learning model. The user will get the predicted values and with its reference the user can decide to book their tickets accordingly. In the current day scenario flight companies try to manipulate the flight ticket prices to maximize their profits. There are many people who travel regularly through flights and so they have an idea about the best time to book cheap tickets. But there are also many people who are inexperienced in booking tickets and end up falling in discount traps made by the companies where actually they end up spending more than they should have. The proposed system can help save millions of rupees of customers by proving them the information to book tickets at the right time. The proposed project title is “Air Fare prediction system”.

1.2 Motivation:

In the current day scenario flight companies try to manipulate the flight ticket prices to maximize their profits. There are many people who travel regularly through flights and so they have an idea about the best time to book cheap tickets. But there are also many people who are inexperienced in booking tickets and end up falling in discount traps made by the companies where actually they end up spending more than they should have. The proposed system can help save millions of rupees of customers by proving them the information to book tickets at the right time.

1.3 Problem Statement:

Now a days, Travelling has been increased in large volume. Machine Learning algorithms and statistics are used to predict the price of air fare with the help of current and past data. Machine learning techniques helps the customers to predict early stage for the pricing of the tickets.

1.4 Objectives:

Generally flight rates vary widely, it is difficult to predict the price of a ticket, therefore buying at different periods could result in significant price differences. From the perspective of the customer, models are employed to forecast the cheapest ticket price as well as the best time to purchase a ticket. While airlines work to maintain the highest possible overall revenue and optimise their profit, customers are seeking for all-time low costs. Airlines employ a variety of strategies, such as demand forecasting and pricing discrimination, to boost their profits. With the assumption that these characteristics affect machine learning model that distinguish flight prices is chosen for this purpose, and the performance of the models is tested. The purpose is to study the factors that influence the flight prices. Then use the information to build a system to predict the flight prices.

1.5 Scope:

In order to meet the demands of humanity in the face of growing population, travelling is increased daily through the air lines. While airlines work to maintain the highest possible overall revenue and optimise their profit, customers are seeking for all-time low costs.

1.6 Random Forest Algorithm:

Random Forest is a popular machine learning algorithm that belongs to the supervised learning technique. It can be used for both Classification and Regression problems in ML. It is based on the concept of ensemble learning, which is a process of combining multiple classifiers to solve a complex problem and to improve the performance of the model.

As the name suggests, "Random Forest is a classifier that contains a number of decision trees on various subsets of the given dataset and takes the average to improve the predictive accuracy of that dataset." Instead of relying on one decision tree, the random forest takes the prediction from each tree and based on the majority votes of predictions, and it predicts the final output.

The greater number of trees in the forest leads to higher accuracy and prevents the problem of overfitting**.** The Figure 1.1 explains the working of the Random Forest algorithm:

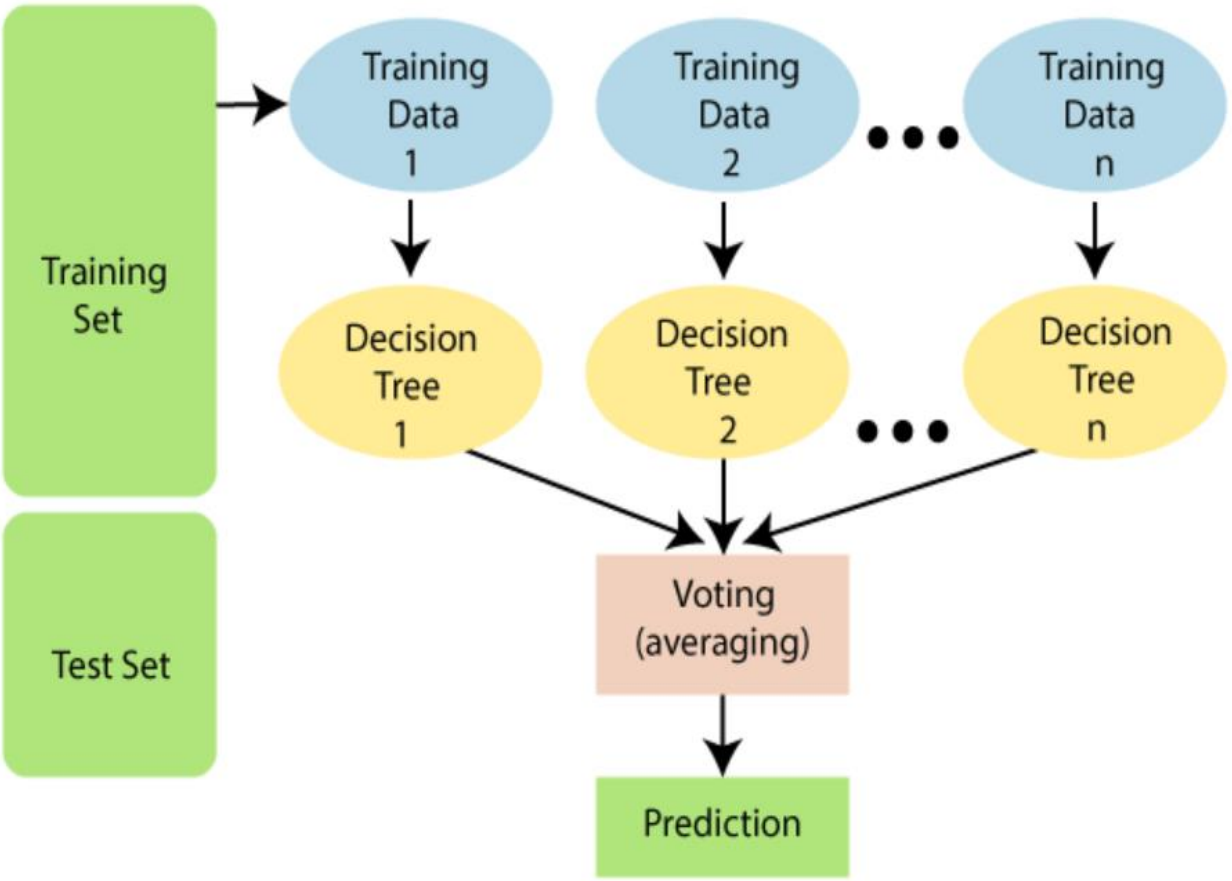


Figure 1.1 Random Forest Algorithm

Since the random forest combines multiple trees to predict the class of the dataset, it is possible that some decision trees may predict the correct output, while others may not. But together, all the trees predict the correct output. Therefore, below are two assumptions for a better Random forest classifier they are There should be some actual values in the feature variable of the dataset so that the classifier can predict accurate results rather than a guessed result, The predictions from each tree must have very low correlations.

Below are some points that explain why we should use the Random Forest algorithm:

- It takes less training time as compared to other algorithms.
- It predicts output with high accuracy, even for the large dataset it runs efficiently.
- It can also maintain accuracy when a large proportion of data is missing.

1.7 Google Colab:

Google Colab (short for Collaboratory) is a free, cloud-based platform for working with Python that allows you to write, run, and share code, as well as work collaboratively on projects with others.

Colab is built on top of Jupyter Notebook, a popular interactive coding environment, and allows users to easily create and share notebooks containing code, text, and multimedia content. Colab is fully integrated with Google Drive, so you can easily save and access your notebooks and data files from anywhere. One of the biggest advantages of using Colab is that it provides free access to powerful computing resources, including GPUs and TPUs, which are typically very expensive to use. This makes it an ideal platform for training and testing deep learning models and other computationally intensive tasks.

Colab also comes with a wide range of pre-installed libraries and tools, such as TensorFlow, NumPy, and pandas, making it easy to get started with machine learning and data analysis.

CHAPTER 2

SYSTEM ANALYSIS

The Flight fare prediction system has mainly 4 steps.

1. Exploratory Data Analysis.
2. Feature Extraction.
3. Algorithm.
4. Hyper Parameter Tuning.

These are the stages that help our System to stand apart from the other existing systems in the market.

2.1 Existing System:

There are several studies and work carried out earlier. Most of them are developed by major companies. There is a lot of demand for these software, hence companies are trying their hands in this field. Some of the existing systems include software like

- Agoda
- Skyscanner
- Kayak
- Hitlist
- Hipmunk
- Momondo
- Hopper
- Ixigo
- Goibibo

These are some of the existing systems that does the same task, but most of these are based on the idea of making profit rather than focussing more on service. They provide the same results but are commercial, so common people can’t use these. Even though some of these are not commercial their accuracy is far low compared to our system hence the proposed system to overcome this problem will be as follows.

2.2 Proposed System:

We are proposing a system that helps the user to predict the price of an airline ticket with optimum accuracy. Firstly, the user needs to fill the required input fields provided on the webpage. The input fields include the information about the date of the journey i.e., the date of departure and the departure time suitable for the user to start his flight.

Up next, the user needs to select the arrival time. Source and destination are to be chosen by the user from the dropdown menu linked to the input field. Later, he/she has to select the number of halts in the journey which will impact the cost of the ticket.

Lastly, the most important factor is the choice of the airline company that the user chooses to travel with. A dropdown menu is attached to for the same. Upon providing all the input fields, and clicking the 'Submit" button, the system enables the user to predict the price of the airline ticket.

2.3 Requirement Analysis:

The Requirement Analysis consists of an analysis of different types of requirements such as Functional Requirements, Non-Functional Requirements, Hardware Requirements and software Requirements. These features, called requirements, must be quantifiable, relevant and detailed.

2.3.1 Functional Requirements:

Functional requirement refers to the functionalities that are applicable to a system. It refers to the product features or functions that developers must implement to enable users to accomplish their tasks. So, it's important to make them clear both for the development team and the stakeholders.

Hence in our case the system must be able to:

1. Let the user Login first.
2. Store the details of the user if he/she is a first time user.
3. Give a result about Login request and redirect to main features page.
4. Accept input about journey from the user.
5. Predict the price upon submission.
6. Display the result on the screen.

2.3.2 Non-Functional Requirements:

A non-functional requirement is a system must behave or how the system behavior is. This also specifies how the system's quality characteristics or quality attributes. In order to put this constraint upon the specific system behavior, the qualities goals of the designed system should go in these:

Usability:

The usability of the system must be simple and easy. It should be capable of having minimum interaction with the user, but at the same time provide the best results possible. It must be easy to use.

Efficiency:

The system must be efficient to use, it should provide accurate results and predict the price, in all cases. It must work efficiently.

Required User Ability:

As working with Artificial Intelligence models requires some previous knowledge, the system should be designed in such a way that even people with minimal knowledge about thedomain and its reaches should be capable of using it. Users should require minimal knowledge to work with this system.

Reliability:

The system must be reliable, it should not lead to unnecessary crashes and stops Even though any kind of errors occur, it should have good exception handling mechanisms.

Security:

Preventive measures have to be taken to ensure that the data of the registered users and the data sets present in the storage units must not be allowed to be accessed by anyone. Authentication methods have to be established so that only those who have the permissions are capable of accessing the data sets and processing them.

Along with providing the functional requirements that the user expects the system to perform, certain requirements such as the above ones are to be met in order to grant the user a friendlier experience in handling and using the system.

2.3.3 Hardware Requirements

This contains the list of the basic hardware requirements the system must satisfy in order to run and execute the model perfectly.

- 1.6 GHz or faster processor.
- 2048 MB RAM (1.5 GB if running on a virtual machine).
- 3 GB of available hard-disk space.
- 6400 RPM hard-disk drive.
- DirectX 11-capable video card running at 1920 x 1080 or higher display resolution.
- Ethernet RJ-45
- WiFi-5/6

2.3.4 Software Requirements

This contains the list of the basic software requirements the system must satisfy in order to run and execute the model perfectly.

- Kaggle Dataset
- Matplotlib
- SciKit-Learn
- Keras
- NumPy
- Pandas
- Python Pickle
- Jupyter Notebook
- Google Colaboratory
- Visual Studio Code
- MySQL
- Python Flask
- Python pip

CHAPTER 3
SYSTEM DESIGN

The System design is the process of defining the architecture, modules, interfaces, and data for a system to satisfy specified requirements. The whole process has been divided into four stages i.e.

1. Exploratory Data Analysis
2. Feature Engineering
3. Feature selection
4. Model Deployment

3.1 System Architecture

System architecture is the conceptual model that defines the structure, behaviour and views of a system. The diagram in Figure 3.1 is an architectural design for the Airfare Prediction System. This system reads input from the user, process it and predict the flight price for that travel.

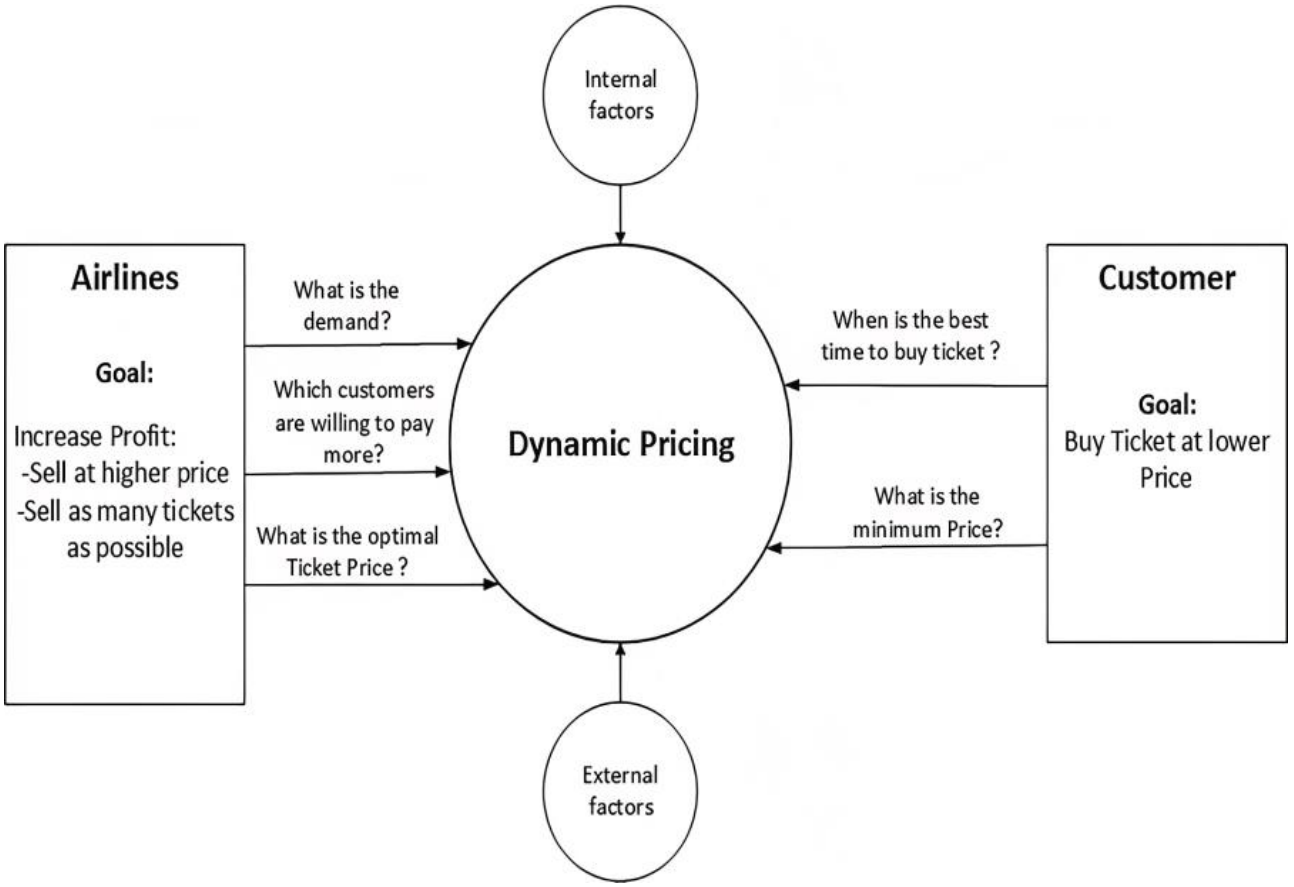


Figure 3.1 System Architecture for Airfare Prediction System

3.2 Data Flow Diagram

Data flow diagrams are used to graphically represent the flow of data in a business- information system. DFD describes the processes that are involved in a system to transfer data from the input to the file storage and report generation.

Data flow diagrams can be divided into logical and physical. The logical data flow diagram describes the flow of data through a system to perform certain functions of a business. The physical data flow diagram describes the implementation of the logical data flow. Figure 3.2 shows the general processes of the Airfare Prediction System that represents the overall flow of the data in the system.

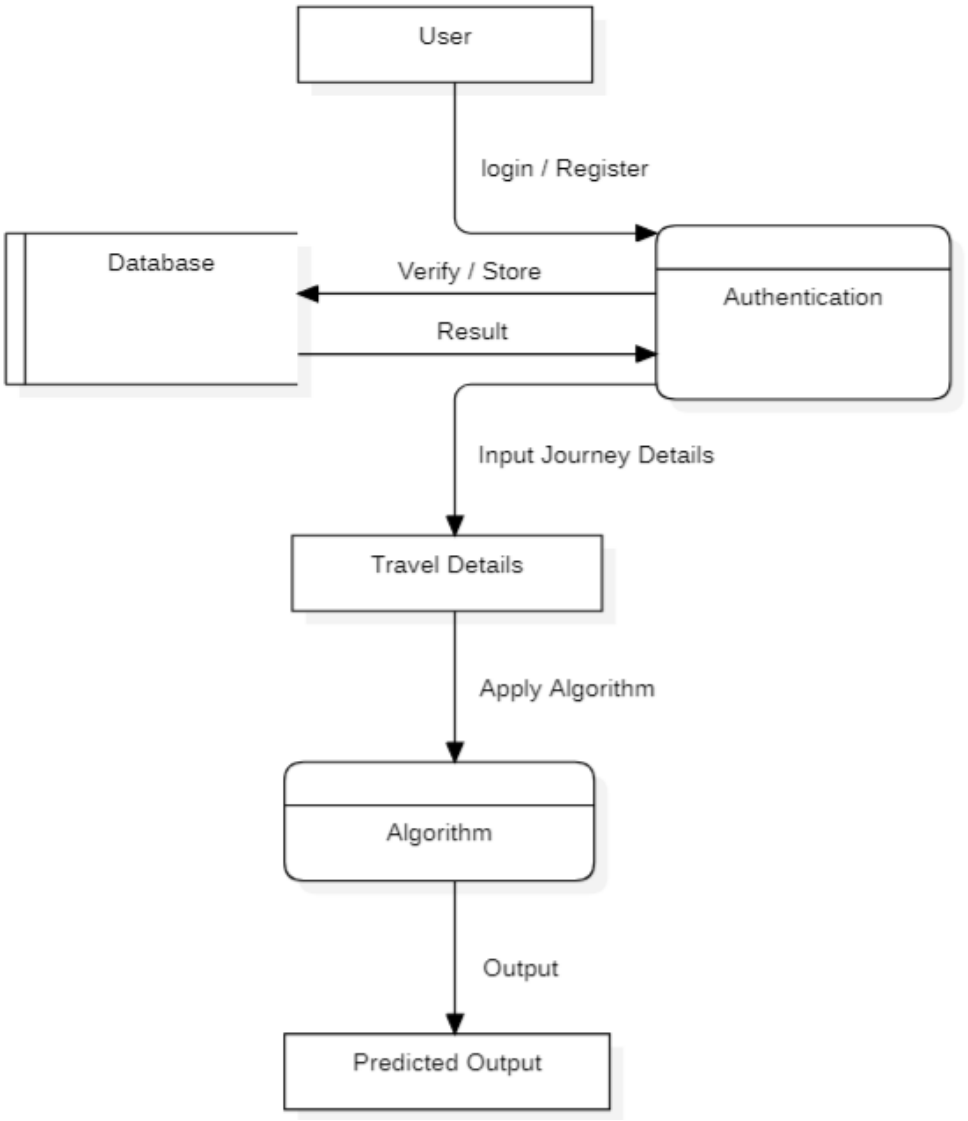


Figure 3.2 Data Flow Diagram for Airfare Prediction System

3.3 Use Case Diagram

Use case diagrams are usually referred to as behavior diagrams used to describe a set of actions (use cases) that some system or systems (subject) should or can perform in collaboration with one or more external users of the system (actors). A use case is a methodology used in system analysis to identify, clarify and organize system requirements.

The use case should contain all system activities that have significance to the users. A use case can be thought of as a collection of possible scenarios related to a particular goal, indeed, the use case and goal are sometimes considered to be synonymous. The main purpose of a use case diagram is to show what system functions are performed for which actor. Figure 3.3 represents the use case diagram that shows the behavior and actions that are performed by the general Airfare Prediction System.

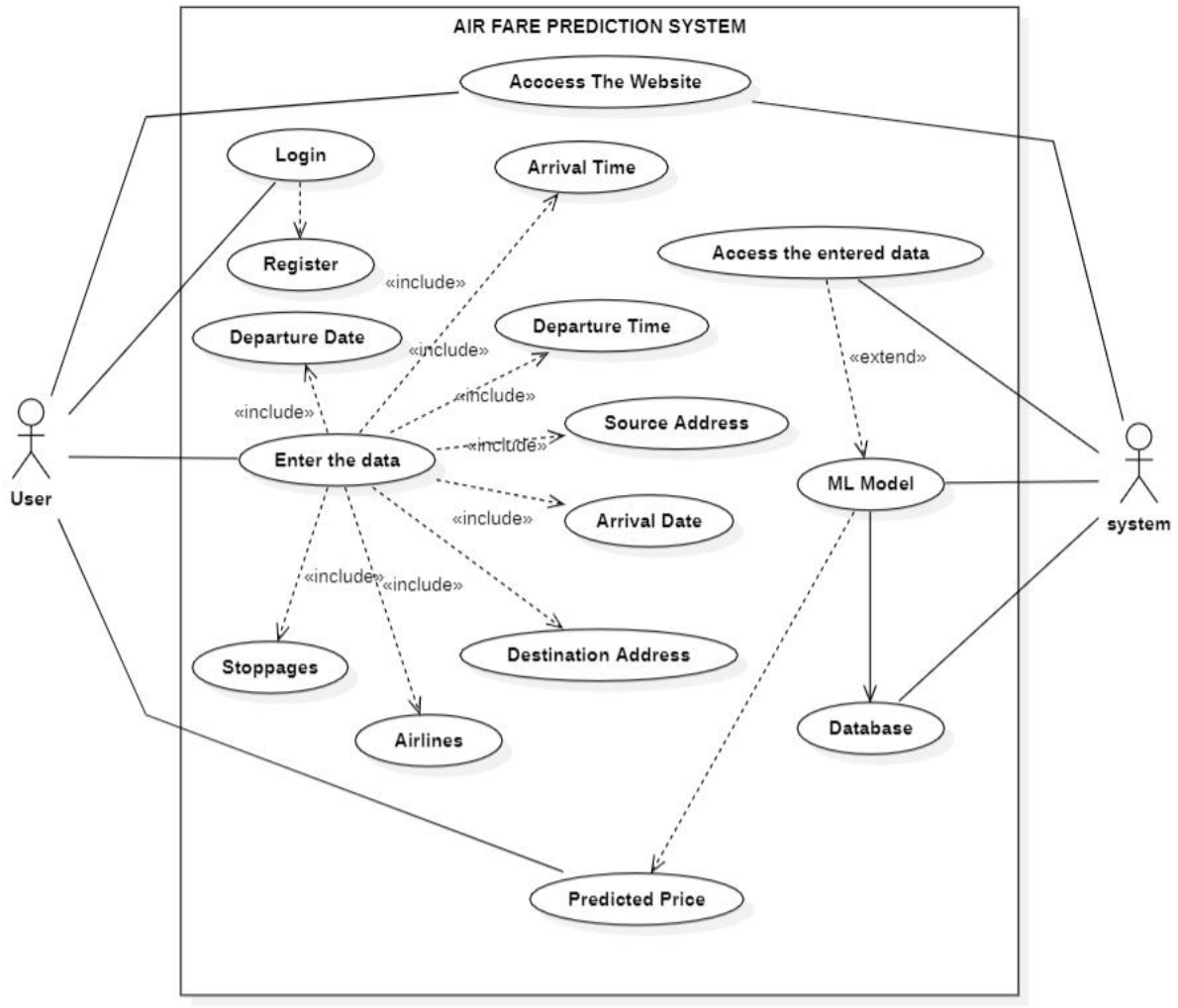


Figure 3.3 Use Case Diagram for Airfare Prediction System

3.4 Sequence Diagram

A sequence diagram shows object interactions arranged in a time sequence. It depicts the objects and classes involved in the scenario and the sequence of messages exchanged between the objects needed to carry out the functionality of the scenario.

They capture the interaction between objects in the context of a collaboration. Sequence Diagrams are time focused, and they show the order of the interaction visually by using the vertical axis of the diagram to represent time, what messages are sent and when.

It captures:

- The interaction that takes place in a collaboration that either realizes a use case or an operation (instance diagrams or generic diagrams)
- High-level interactions between users of the system and the system, between the system and other systems, or between subsystems (also known as systemsequence diagrams).

The activities that are involved in the Airfare Prediction System are arranged in a time sequence and theinteractions between the objects are shown in Figure 3.4.

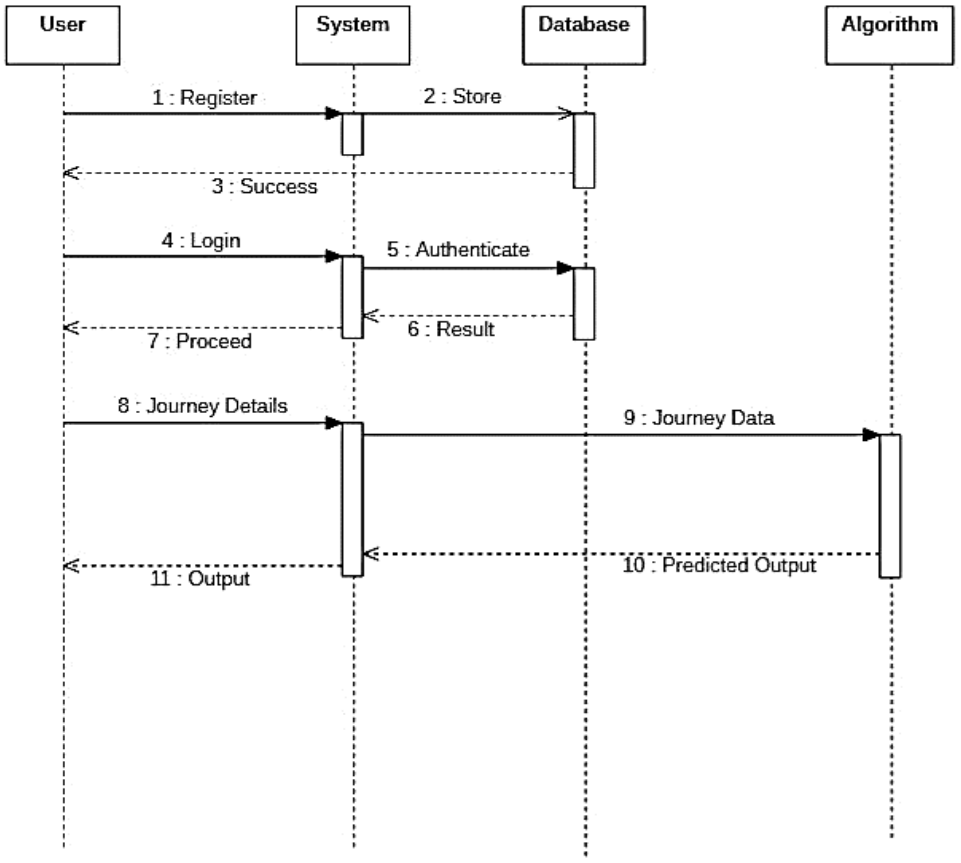


Figure 3.4 Sequence Diagram for Airfare Prediction System

3.5 Class Diagram

The class diagram is a static diagram. It represents the static view of an application. The class diagram is not only used for visualizing, describing, and documenting different aspects of a system but also for constructing executable code of the software application.

A class diagram describes the attributes and operations of a class and also the constraints imposed on the system. The class diagrams are widely used in the modelling of object-oriented systems because they are the only UML diagrams, which can be mapped directly with object-oriented languages.

The class diagram shows a collection of classes, interfaces, associations, collaborations, and constraints. It is also known as a structural diagram. Figure 3.5 shows the overall structure of the Airfare Prediction system that represents all the classes, attributes, and methods involved in the System.

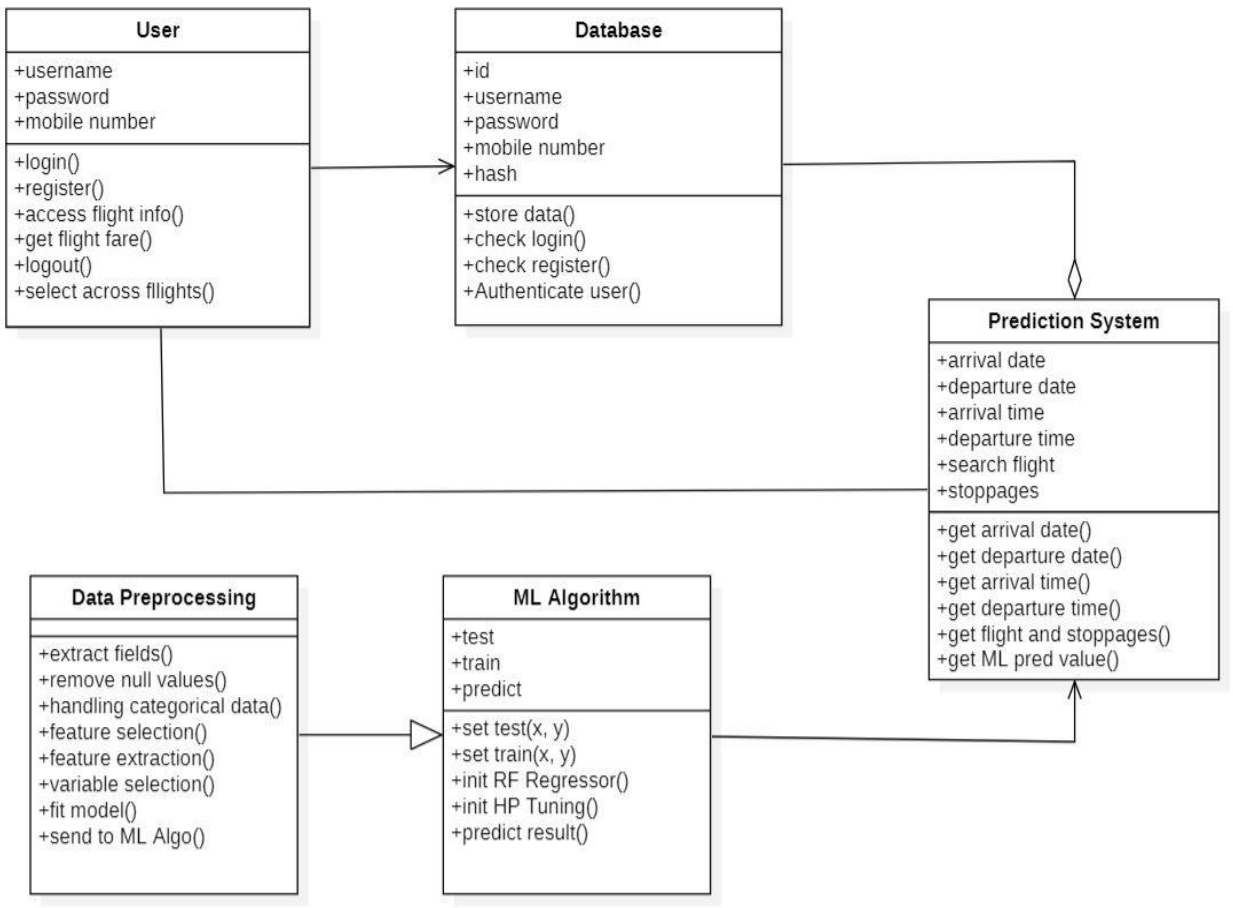


Figure 3.5 Class Diagram for Airfare Prediction System

3.6 Activity Diagram

An activity diagram is basically a flowchart to represent the flow from one activity to another activity. The activity can be described as an operation of the system. The control flow is drawn from one operation to another. This flow can be sequential, branched, or concurrent. The flow carries from the starting of an activity to the end of an activity, it performs different types of operations through the flow that are shown in Figure 3.6.

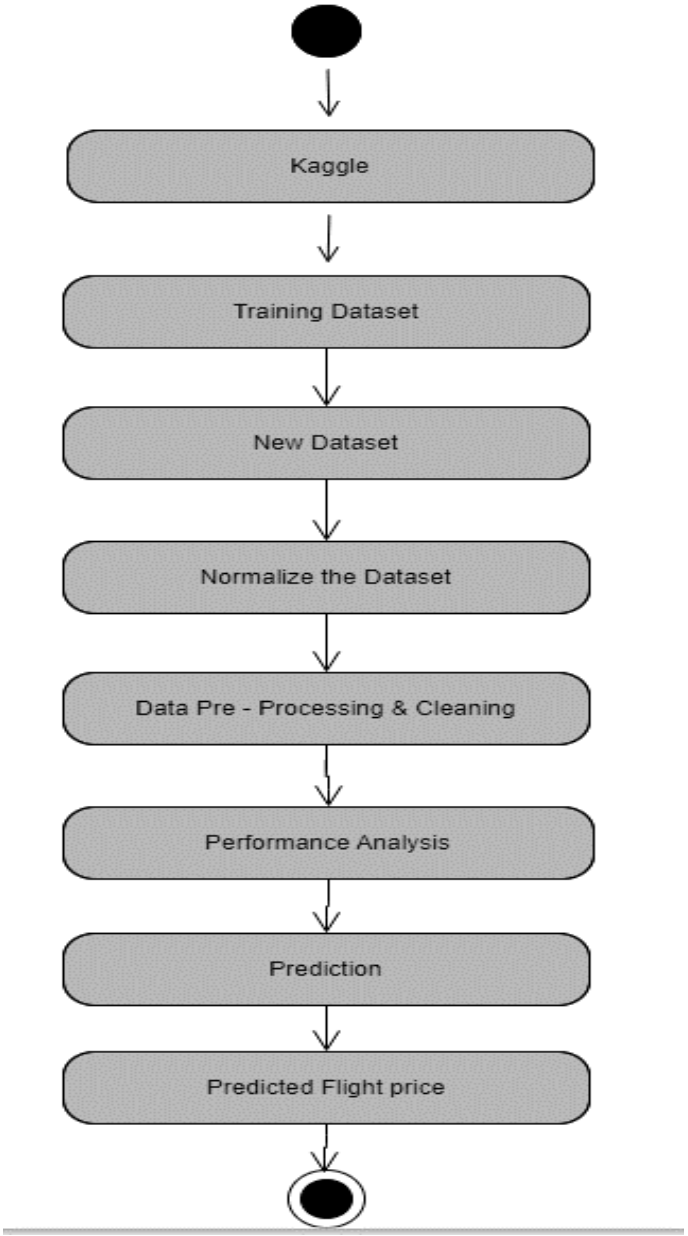


Figure 3.6 Activity Diagram for Airfare Prediction System

CHAPTER 4
SYSTEM IMPLEMENTATION

The proposed system has various modules in which the Flight Fare Prediction system implements essential steps to train, extract, fit and predict the output of flight ticket fare. Flight Fare Prediction System takes a long duration to train the Random Forest model. The system takes hundreds of steps to train the model. The proposed system uses.

4.1 Setup Prerequisites

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

sns.set()
```

The above code from the Google Collaboratory imports the necessary modules of Python like NumPy, pandas, matplotlib and seaborn to make use of them in EDA.

4.2 Downloading The Dataset

The dataset of flight fare prediction system is available in Kaggle datasets which is made in consideration of the details of Indian Airlines as shown in the Figure 4.1.

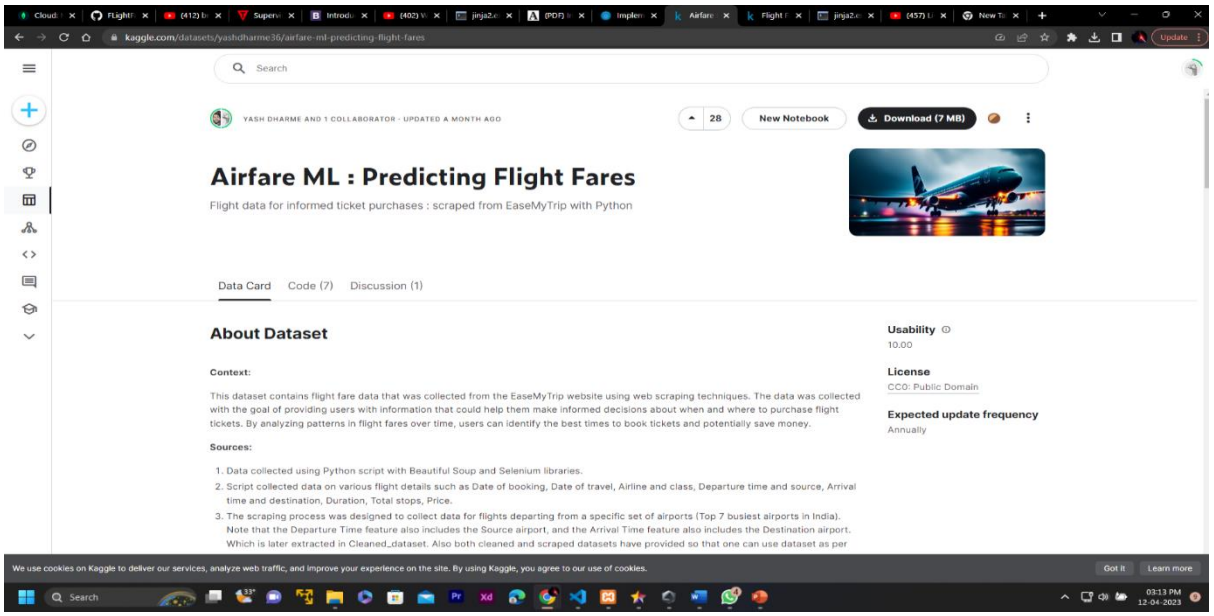


Figure 4.1 kaggle Datasets Home page

There are some columns which are essential for the exploratory data analysis. There are approximately 10000 rows of data entries in the dataset as shown in the Figure 4.2. Their head details are shown in the dataset columnar header.

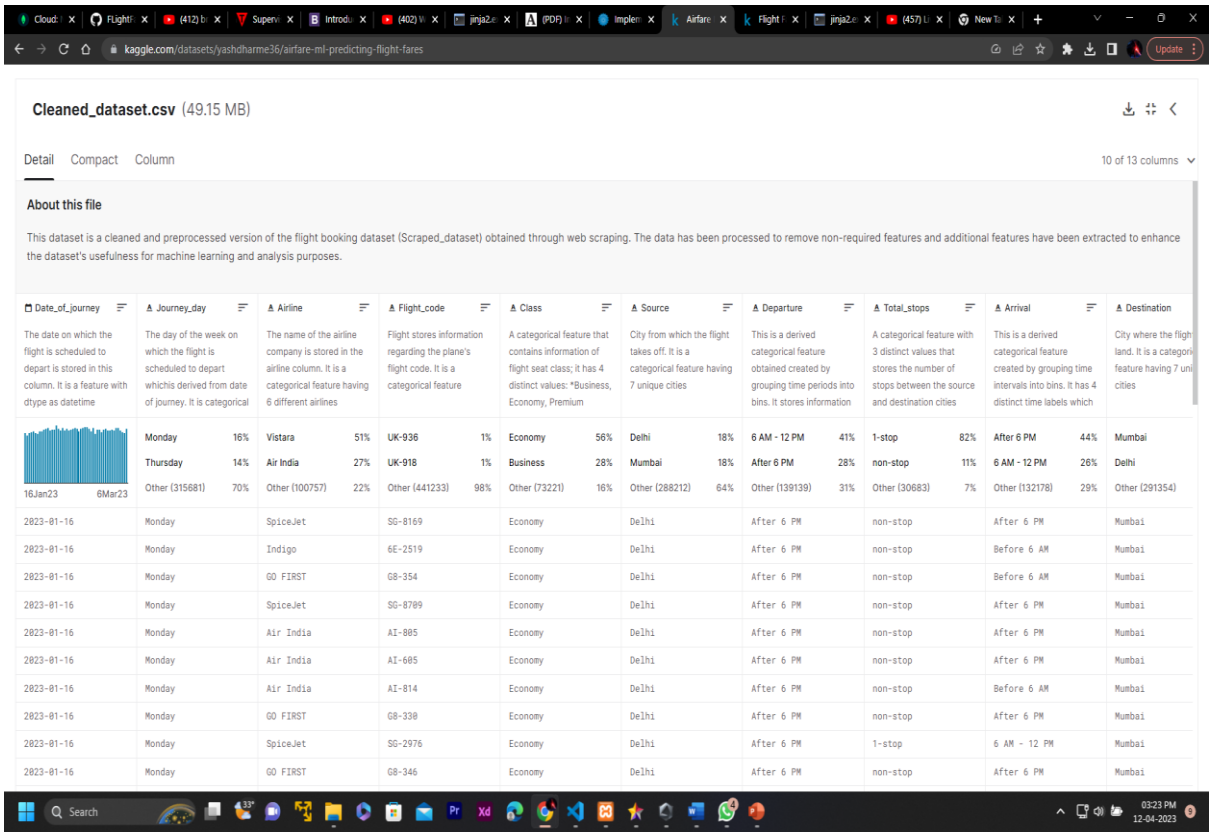


Figure 4.2 Flight Fare Dataset

4.3 Importing The Dataset

```
train_data = pd.read_excel(r"/content/drive/MyDrive/FlightFare/Data_Train.xlsx")
```

The block of code imports the Google Drive and mount it to the Colab to use it and creates an instance with virtual memory and storage space to work with the files in it.

```
pd.set_option('display.max_columns', None)
train_data.head()
```

Here, we are importing the dataset and reading its head values to make the top 5 columns visible in the cell.

	Airline	Date_of_Journey	Source	Destination	Route	Dep_Time	Arrival_Time	Duration	Total_Stops	Additional_Info	Price
0	IndiGo	24/03/2019	Banglore	New Delhi	BLR → DEL	22:20	01:10 22 Mar	2h 50m	non-stop	No info	3897
1	Air India	1/05/2019	Kolkata	Banglore	CCU → IXR → BBI → BLR	05:50	13:15	7h 25m	2 stops	No info	7662
2	Jet Airways	9/06/2019	Delhi	Cochin	DEL → LKO → BOM → COK	09:25	04:25 10 Jun	19h	2 stops	No info	13882
3	IndiGo	12/05/2019	Kolkata	Banglore	CCU → NAG → BLR	18:05	23:30	5h 25m	1 stop	No info	6218
4	IndiGo	01/03/2019	Banglore	New Delhi	BLR → NAG → DEL	16:50	21:35	4h 45m	1 stop	No info	13302

This is the output of dataset showing top 5 rows in the ordered manner.

```
train_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10683 entries, 0 to 10682
Data columns (total 11 columns):
 #   Column           Non-Null Count  Dtype 
---  ------           --------------  ----- 
 0   Airline          10683 non-null  object
 1   Date_of_Journey  10683 non-null  object
 2   Source           10683 non-null  object
 3   Destination      10683 non-null  object
 4   Route            10682 non-null  object
 5   Dep_Time         10683 non-null  object
 6   Arrival_Time     10683 non-null  object
 7   Duration         10683 non-null  object
 8   Total_Stops      10682 non-null  object
 9   Additional_Info  10683 non-null  object
 10  Price            10683 non-null  int64 
dtypes: int64(1), object(10)
memory usage: 918.2+ KB
```

```
train_data["Duration"].value_counts()
```

From description we can see that Date_of_Journey is an object data type. Therefore, we have to convert this datatype into timestamp so as to use this column properly for prediction, because our model will not be able to understand Theses string values, it just understand Time-stamp. For this we require pandas to_datetime to convert object data type to datetime dtype.

4.3 Data Pre-processing

Data pre-processing is an important step in building a machine learning model, especially when dealing with complex data such as flight data. This process should be done before the model building and model training in order to feed the machine learning algorithm to give the output.

```
#remove all null values
train_data.dropna(inplace = True)
```

```
train_data.isnull().sum()
```

```
Airline          0
Date_of_Journey  0
Source           0
Destination      0
Route            0
Dep_Time         0
Arrival_Time     0
Duration         0
Total_Stops      0
Additional_Info  0
Price            0
dtype: int64
```

The above block of code checks if there are any missing values or Null values in the attributes of dataset and gives the sum and count of the Null values that exist in features.

4.4 EDA of Train Dataset

EDA, or exploratory data analysis, is an important step in understanding the data and preparing it for machine learning modeling. EDA involves using visual and statistical methods to explore the data, identify patterns and relationships, and identify any issues or anomalies that may need to be addressed.

```
#extract day from dt.day
train_data["Journey_day"] = pd.to_datetime(train_data.Date_of_Journey, format="%d/%m/%Y").dt.day
```

```
#extract month from dt.month
train_data["Journey_month"] = pd.to_datetime(train_data["Date_of_Journey"], format = "%d/%m/%Y").dt.month
```

The above lines of code extracts the date and month from dt.day and dt.month attributes.

```
train_data.head()
```

	Airline	Date_of_Journey	Source	Destination	Route	Dep_Time	Arrival_Time	Duration	Total_Stops	Additional_Info	Price	Journey_day	Journey_month
0	IndiGo	24/03/2019	Banglore	New Delhi	BLR → DEL	22:20	01:10 22 Mar	2h 50m	non-stop	No info	3897	24	3
1	Air India	1/05/2019	Kolkata	Banglore	CCU → IXR → BBI → BLR	05:50	13:15	7h 25m	2 stops	No info	7662	1	5
2	Jet Airways	9/06/2019	Delhi	Cochin	DEL → LKO → BOM → COK	09:25	04:25 10 Jun	19h	2 stops	No info	13882	9	6
3	IndiGo	12/05/2019	Kolkata	Banglore	CCU → NAG → BLR	18:05	23:30	5h 25m	1 stop	No info	6218	12	5
4	IndiGo	01/03/2019	Banglore	New Delhi	BLR → NAG → DEL	16:50	21:35	4h 45m	1 stop	No info	13302	1	3

```
# Since we have converted Date_of_Journey column into integers, Now we can drop as it is of no use.
train_data.drop(["Date_of_Journey"], axis = 1, inplace = True)
```

```
#Departure time
# Extracting Hours
train_data["Dep_hour"] = pd.to_datetime(train_data["Dep_Time"]).dt.hour

# Extracting Minutes
train_data["Dep_min"] = pd.to_datetime(train_data["Dep_Time"]).dt.minute

# drop Dep_Time as it is of no use
train_data.drop(["Dep_Time"], axis = 1, inplace = True)
```

```
# Arrival time
# Extracting Hours
train_data["Arrival_hour"] = pd.to_datetime(train_data.Arrival_Time).dt.hour

# Extracting Minutes
train_data["Arrival_min"] = pd.to_datetime(train_data.Arrival_Time).dt.minute

#drop Arrival_Time as it is of no use
train_data.drop(["Arrival_Time"], axis = 1, inplace = True)
```

Extracting the arrival hours , arrival minutes, departure hours and departure minutes features from the existing Arrival time and Departure time attributes and dropping the Time features further due to no use and significance of features.

```
# Assigning and converting Duration column into list
duration = list(train_data["Duration"])

for i in range(len(duration)):
    if len(duration[i].split()) != 2:    # Check if duration contains only hour or mins
        if "h" in duration[i]:
            duration[i] = duration[i].strip() + " 0m"   # Adds 0 minute
        else:
            duration[i] = "0h " + duration[i]           # Adds 0 hour

duration_hours = []
duration_mins = []
for i in range(len(duration)):
    duration_hours.append(int(duration[i].split(sep = "h")[0]))    # Extract hours from duration
    duration_mins.append(int(duration[i].split(sep = "m")[0].split()[-1]))   # Extracts only minutes from duration
```

Extracting the hours and minutes features from the Duration attribute so that the hour and minute gets initiated with 0.

```
train_data["Duration_hours"] = duration_hours
train_data["Duration_mins"] = duration_mins
```

```
train_data.drop(["Duration"], axis = 1, inplace = True)
```

Dropping the duration feature from the train dataset and displaying the head() data from the train dataset. It would display top five rows/ data entries.

```
train_data.head()
```

Airline	Source	Destination	Route	Total_Stops	Additional_Info	Price	Journey_day	Journey_month	Dep_hour	Dep_min	Arrival_hour	Arrival_min	Duration_hours	Duratio
IndiGo	Banglore	New Delhi	BLR → DEL	non-stop	No info	3897	24	3	22	20	1	10	2	
Air India	Kolkata	Banglore	CCU → IXR → BBI → BLR	2 stops	No info	7662	1	5	5	50	13	15	7	
Jet Airways	Delhi	Cochin	DEL → LKO → BOM → COK	2 stops	No info	13882	9	6	9	25	4	25	19	
IndiGo	Kolkata	Banglore	CCU → NAG → BLR	1 stop	No info	6218	12	5	18	5	23	30	5	
IndiGo	Banglore	New Delhi	BLR → NAG → DEL	1 stop	No info	13302	1	3	16	50	21	35	4	

4.5 Handling The Categorical Data

Categorical data refers to data that represents categories or labels rather than numerical values. This type of data is often qualitative in nature and can be represented as strings or text.

Handling categorical data is an important step in data pre-processing for machine learning models. There are two main techniques for handling categorical data: label encoding and one-hot encoding.

From the above features, these are the Categorical Features:

- Airline
- Source
- Destination
- Route
- Total_Stops
- Additional_Info

```
train_data["Airline"].value_counts()
```

```
Jet Airways                          3849
IndiGo                               2053
Air India                            1751
Multiple carriers                    1196
SpiceJet                              818
Vistara                               479
Air Asia                              319
GoAir                                 194
Multiple carriers Premium economy      13
Jet Airways Business                    6
Vistara Premium economy                 3
Trujet                                  1
Name: Airline, dtype: int64
```

```
# From graph we can see that Jet Airways have the highest Price.

# Airline vs Price
sns.catplot(y = "Price", x = "Airline", data = train_data.sort_values("Price", ascending = False), kind="boxen", height = 6, aspect = 3)
plt.show()
```

This block gives the graph plotted for Airline vs Price with the help of Seaborn Visualisation library.

```
#One hot encoding
Airline = train_data[["Airline"]]

Airline = pd.get_dummies(Airline, drop_first= True)

Airline.head()
```

	Airline_Air India	Airline_GoAir	Airline_IndiGo	Airline_Jet Airways	Airline_Jet Airways Business	Airline_Multiple carriers	Airline_Multiple carriers Premium economy	Airline_SpiceJet	Airline_Trujet	Airline_Vistara	Airline_Vistara Premium economy
0	0	0	1	0	0	0	0	0	0	0	0
1	1	0	0	0	0	0	0	0	0	0	0
2	0	0	0	1	0	0	0	0	0	0	0
3	0	0	1	0	0	0	0	0	0	0	0
4	0	0	1	0	0	0	0	0	0	0	0

One hot encoding is a technique for handling categorical data in which each unique category in a feature is represented as a binary feature. Each binary feature represents whether the original feature had that particular category or not. It is done to feed the labelled data to model.

```
train_data["Source"].value_counts()
```

```
Delhi       4536
Kolkata     2871
Banglore    2197
Mumbai       697
Chennai      381
Name: Source, dtype: int64
```

```
# Source vs Price

sns.catplot(y = "Price", x = "Source", data = train_data.sort_values("Price", ascending = False), kind="boxen", height = 4, aspect = 3)
plt.show()
```

Plotting the graph for Source vs Price with the Seaborn Library.

```
#one hot encoding
Source = train_data[["Source"]]

Source = pd.get_dummies(Source, drop_first= True)

Source.head()
```

	Source_Chennai	Source_Delhi	Source_Kolkata	Source_Mumbai
0	0	0	0	0
1	0	0	1	0
2	0	1	0	0
3	0	0	1	0
4	0	0	0	0

```
train_data["Destination"].value_counts()
```

```
Cochin       4536
Banglore     2871
Delhi        1265
New Delhi     932
Hyderabad     697
Kolkata       381
Name: Destination, dtype: int64
```

```
#one hot encoding
Destination = train_data[["Destination"]]

Destination = pd.get_dummies(Destination, drop_first = True)

Destination.head()
```

	Destination_Cochin	Destination_Delhi	Destination_Hyderabad	Destination_Kolkata	Destination_New Delhi
0	0	0	0	0	1
1	0	0	0	0	0
2	1	0	0	0	0
3	0	0	0	0	0
4	0	0	0	0	1

Again performing the One Hot Encoding for both the source and destination features to get the indexed and labelled binary data in the form of matrix and vector.

```
train_data["Route"]
```

```
0                  BLR → DEL
1          CCU → IXR → BBI → BLR
2          DEL → LKO → BOM → COK
3                CCU → NAG → BLR
4                BLR → NAG → DEL
                 ...
10678              CCU → BLR
10679              CCU → BLR
10680              BLR → DEL
10681              BLR → DEL
10682    DEL → GOI → BOM → COK
Name: Route, Length: 10682, dtype: object
```

```
#drop column "Route" and "Additional_Info" as it is of no use
train_data.drop(["Route", "Additional_Info"], axis = 1, inplace = True)
```

```
train_data["Total_Stops"].value_counts()
```

```
1 stop      5625
non-stop    3491
2 stops     1520
3 stops       45
4 stops        1
Name: Total_Stops, dtype: int64
```

The features like Route and Additional_info are dropped here because of its no usability.

```
#Label encoding
train_data.replace({"non-stop": 0, "1 stop": 1, "2 stops": 2, "3 stops": 3, "4 stops": 4}, inplace = True)
```

```
train_data.head()
```

	Airline	Source	Destination	Total_Stops	Price	Journey_day	Journey_month	Dep_hour	Dep_min	Arrival_hour	Arrival_min	Duration_hours	Duration_mins
0	IndiGo	Banglore	New Delhi	0	3897	24	3	22	20	1	10	2	50
1	Air India	Kolkata	Banglore	2	7662	1	5	5	50	13	15	7	25
2	Jet Airways	Delhi	Cochin	2	13882	9	6	9	25	4	25	19	0
3	IndiGo	Kolkata	Banglore	1	6218	12	5	18	5	23	30	5	25
4	IndiGo	Banglore	New Delhi	1	13302	1	3	16	50	21	35	4	45

Label encoding is a technique for handling categorical data in which each unique category in a feature is assigned a numerical value. This is typically done using a mapping dictionary that maps each category to a numerical value. It is done to the features because of its ordered data.

```
#concatenate all dataframes
data_train = pd.concat([train_data, Airline, Source, Destination], axis = 1)
```

```
data_train.drop(["Airline", "Source", "Destination"], axis = 1, inplace = True)
```

```
data_test.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2671 entries, 0 to 2670
Data columns (total 28 columns):
 #   Column                            Non-Null Count  Dtype
---  ------                            --------------  -----
 0   Total_Stops                       2671 non-null   int64
 1   Journey_day                       2671 non-null   int64
 2   Journey_month                     2671 non-null   int64
 3   Dep_hour                          2671 non-null   int64
 4   Dep_min                           2671 non-null   int64
 5   Arrival_hour                      2671 non-null   int64
 6   Arrival_min                       2671 non-null   int64
 7   Duration_hours                    2671 non-null   int64
 8   Duration_mins                     2671 non-null   int64
 9   Air India                         2671 non-null   uint8
 10  GoAir                             2671 non-null   uint8
 11  IndiGo                            2671 non-null   uint8
 12  Jet Airways                       2671 non-null   uint8
 13  Jet Airways Business              2671 non-null   uint8
 14  Multiple carriers                 2671 non-null   uint8
 15  Multiple carriers Premium economy 2671 non-null   uint8
 16  SpiceJet                          2671 non-null   uint8
 17  Vistara                           2671 non-null   uint8
 18  Vistara Premium economy           2671 non-null   uint8
 19  Chennai                           2671 non-null   uint8
 20  Delhi                             2671 non-null   uint8
 21  Kolkata                           2671 non-null   uint8
 22  Mumbai                            2671 non-null   uint8
 23  Cochin                            2671 non-null   uint8
 24  Delhi                             2671 non-null   uint8
 25  Hyderabad                         2671 non-null   uint8
 26  Kolkata                           2671 non-null   uint8
 27  New Delhi                         2671 non-null   uint8
dtypes: int64(9), uint8(19)
```

Concatenating all the dataframes like Airlines, source and destination in the Train_data. displaying all the features in the dataset after cleaning and dropping all the null and trivial entries.

```
data_train.head()
```

Arrival_hour	Arrival_min	Duration_hours	Duration_mins	Airline_Air India	Airline_GoAir	Airline_IndiGo	Airline_Jet Airways	Airline_Jet Airways Business	Airline_Multiple carriers	Airline_Multiple carriers Premium economy	Airline_SpiceJet	Ai
1	10	2	50	0	0	1	0	0	0	0	0	
13	15	7	25	1	0	0	0	0	0	0	0	
4	25	19	0	0	0	0	1	0	0	0	0	
23	30	5	25	0	0	1	0	0	0	0	0	
21	35	4	45	0	0	1	0	0	0	0	0	

4.6 Test Dataset

The test dataset is loaded into the Colab from the google drive named Test_set.xlsx for the further training purpose and performing the same procedure till we get clean and useful data.

```
test_data = pd.read_excel(r"/content/drive/MyDrive/FlightFare/Test_set.xlsx")
```

```
print(test_data.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2671 entries, 0 to 2670
Data columns (total 10 columns):
 #   Column           Non-Null Count  Dtype 
---  ------           --------------  ----- 
 0   Airline          2671 non-null   object
 1   Date_of_Journey  2671 non-null   object
 2   Source           2671 non-null   object
 3   Destination      2671 non-null   object
 4   Route            2671 non-null   object
 5   Dep_Time         2671 non-null   object
 6   Arrival_Time     2671 non-null   object
 7   Duration         2671 non-null   object
 8   Total_Stops      2671 non-null   object
 9   Additional_Info  2671 non-null   object
dtypes: object(10)
memory usage: 208.8+ KB
None
```

```
test_data.dropna(inplace = True)
print(test_data.isnull().sum())
```

```
Airline            0
Date_of_Journey    0
Source             0
Destination        0
Route              0
Dep_Time           0
Arrival_Time       0
Duration           0
Total_Stops        0
Additional_Info    0
dtype: int64
```

4.7 EDA of Test Dataset

We will pre-process our dataset. We will extract day and month from the column “Date of Journey” as the model will understand numerical value, for this we will use “pd.to_datetime” for day and month column. “dt.day” and “dt.month” will extract day and month respectively from the given column.

Same process will be doing for the “dep_time” column, “Duration” column and “arrival_time” column and extract hours and min from it. After extracting day, month, hours and min, we will drop “Date of Journey”, “Duration”, “dep_time” & “arrival_time” column from our dataset.

```
# Date_of_Journey
test_data["Journey_day"] = pd.to_datetime(test_data.Date_of_Journey, format="%d/%m/%Y").dt.day
test_data["Journey_month"] = pd.to_datetime(test_data["Date_of_Journey"], format = "%d/%m/%Y").dt.month
test_data.drop(["Date_of_Journey"], axis = 1, inplace = True)
```

```
# Dep_Time
test_data["Dep_hour"] = pd.to_datetime(test_data["Dep_Time"]).dt.hour
test_data["Dep_min"] = pd.to_datetime(test_data["Dep_Time"]).dt.minute
test_data.drop(["Dep_Time"], axis = 1, inplace = True)
```

```
# Arrival_Time
test_data["Arrival_hour"] = pd.to_datetime(test_data.Arrival_Time).dt.hour
test_data["Arrival_min"] = pd.to_datetime(test_data.Arrival_Time).dt.minute
test_data.drop(["Arrival_Time"], axis = 1, inplace = True)
```

```
# Duration
duration = list(test_data["Duration"])

for i in range(len(duration)):
    if len(duration[i].split()) != 2:    # Check if duration contains only hour or mins
        if "h" in duration[i]:
            duration[i] = duration[i].strip() + " 0m"   # Adds 0 minute
        else:
            duration[i] = "0h " + duration[i]           # Adds 0 hour

duration_hours = []
duration_mins = []
for i in range(len(duration)):
    duration_hours.append(int(duration[i].split(sep = "h")[0]))    # Extract hours from duration
    duration_mins.append(int(duration[i].split(sep = "m")[0].split()[-1]))   # Extracts only minutes from duration
```

```
# Adding Duration column to test set
test_data["Duration_hours"] = duration_hours
test_data["Duration_mins"] = duration_mins
test_data.drop(["Duration"], axis = 1, inplace = True)
```

```
#One hot encoding => Airline
print(test_data["Airline"].value_counts())
Airline = pd.get_dummies(test_data["Airline"], drop_first= True)
```

```
Jet Airways                          897
IndiGo                               511
Air India                            440
Multiple carriers                    347
SpiceJet                             208
Vistara                              129
Air Asia                              86
GoAir                                 46
Multiple carriers Premium economy      3
Vistara Premium economy                2
Jet Airways Business                   2
Name: Airline, dtype: int64
```

```
#One hot encoding => Source
print(test_data["Source"].value_counts())
Source = pd.get_dummies(test_data["Source"], drop_first= True)
```

```
Delhi       1145
Kolkata      710
Banglore     555
Mumbai       186
Chennai       75
Name: Source, dtype: int64
```

```
test_data.drop(["Route", "Additional_Info"], axis = 1, inplace = True)
```

```
test_data.replace({"non-stop": 0, "1 stop": 1, "2 stops": 2, "3 stops": 3, "4 stops": 4}, inplace = True)
```

```
print(pd.DataFrame(X_train).head())
```

```
   0  1  2    3  4  5   6  7   8   9   10  11  12  13
0  3  0  5   18  4  8  24  3   2  50  22  20   1  10
1  1  3  0   84  1  8   1  5   7  25   5  50  13  15
2  4  2  1  118  1  8   9  6  19   0   9  25   4  25
3  3  3  0   91  0  8  12  5   5  25  18   5  23  30
4  3  0  5   29  0  8   1  3   4  45  16  50  21  35
```

This is one hot encoded data with the indices and labels with respect to the features in matrix.

```
#concatenate
data_test = pd.concat([test_data, Airline, Source, Destination], axis = 1)
```

```
data_test.drop(["Airline", "Source", "Destination"], axis = 1, inplace = True)
```

```
data_test.head()
```

Journey_day	Journey_month	Dep_hour	Dep_min	Arrival_hour	Arrival_min	Duration_hours	Duration_mins	Air India	GoAir	IndiGo	Jet Airways	Jet Airways Business	Multiple carriers	Multiple carriers Premium economy
6	6	17	30	4	25	10	55	0	0	0	1	0	0	0
12	5	6	20	10	20	4	0	0	0	1	0	0	0	0
21	5	19	15	19	0	23	45	0	0	0	1	0	0	0
21	5	8	0	21	0	13	0	0	0	0	0	0	1	0
24	6	23	55	2	45	2	50	0	0	0	0	0	0	0

Concatenating and dropping the features like Airline, destination and source likewise the train dataset. Now, the test dataset is ready to be fed into the algorithm.

4.8 Feature Extraction

Feature extraction is the process of transforming raw data into a set of features that can be used to represent the data in a machine learning model. The goal of feature extraction is to create a set of features that captures the most relevant information from the raw data and is informative for the machine learning model.

```
data_train.columns
```

```
#Independent variable => Input
X = data_train.loc[:, ['Total_Stops', 'Journey_day', 'Journey_month', 'Dep_hour',
       'Dep_min', 'Arrival_hour', 'Arrival_min', 'Duration_hours',
       'Duration_mins', 'Airline_Air India', 'Airline_GoAir', 'Airline_IndiGo',
       'Airline_Jet Airways', 'Airline_Jet Airways Business',
       'Airline_Multiple carriers',
       'Airline_Multiple carriers Premium economy', 'Airline_SpiceJet',
       'Airline_Trujet', 'Airline_Vistara', 'Airline_Vistara Premium economy',
       'Source_Chennai', 'Source_Delhi', 'Source_Kolkata', 'Source_Mumbai',
       'Destination_Cochin', 'Destination_Delhi', 'Destination_Hyderabad',
       'Destination_Kolkata', 'Destination_New Delhi']]
X.head()
```

Source_Chennai	Source_Delhi	Source_Kolkata	Source_Mumbai	Destination_Cochin	Destination_Delhi	Destination_Hyderabad	Destination_Kolkata	Destination_New Delhi
0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0
0	1	0	0	1	0	0	0	0
0	0	1	0	0	0	0	0	0
0	0	0	0	0	0	0	0	1

```
#Dependent/Target variable => Output
y = data_train.iloc[:, 1]
y.head()
```

```
0     3897
1     7662
2    13882
3     6218
4    13302
Name: Price, dtype: int64
```

Assigning the Dependant and Independent variables by dropping and including the Price feature.

```
plt.figure(figsize = (18,18))
sns.heatmap(train_data.corr(), annot = True, cmap = "RdYlGn")

plt.show()
```

Plotting the graph/ Heat Map for the Correlation between dependant and Independent variables (Price_included and Price_dropped). This is done with the help of Seaborn data visualisation library.

4.9 Using Extra Tree Regressor

The Extra Tree Regressor algorithm is an ensemble of decision trees that are trained using a random subset of features and data samples. This helps to reduce overfitting and increase the model's generalization ability.

```
# Important feature using ExtraTreesRegressor

from sklearn.ensemble import ExtraTreesRegressor
selection = ExtraTreesRegressor()
selection.fit(X, y)
```

```
ExtraTreesRegressor()
```

```
print(selection.feature_importances_)
```

```
[2.38730533e-01 1.42911514e-01 5.44923669e-02 2.39744627e-02
 2.12581648e-02 2.82458996e-02 1.91493429e-02 1.06815953e-01
 1.79610386e-02 9.19199120e-03 1.62601297e-03 1.94659019e-02
 1.40486554e-01 6.69905980e-02 2.00314476e-02 8.91690070e-04
 3.08318394e-03 1.09226695e-04 5.17210083e-03 8.55102617e-05
 4.46140587e-04 9.67948863e-03 3.21351724e-03 8.09596407e-03
 1.10636750e-02 1.51971684e-02 6.21498801e-03 4.52161763e-04
 2.49634036e-02]
```

```
#plot graph of feature

plt.figure(figsize = (12,8))
feat_importances = pd.Series(selection.feature_importances_, index=X.columns)
feat_importances.nlargest(20).plot(kind='barh')
plt.show()
```

Plotting the graphs of features after the feature selection, The graph shows the features vs density.

4.10 Fitting Model With Random Forest Algorithm

The Random Forest algorithm is an ensemble of decision trees, where each tree is trained on a random subset of features and data samples. Random Forest also uses a technique called bootstrapping to generate multiple training datasets from the original data, which helps to reduce overfitting and increase the model's generalization ability.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_state = 42)
```

```
from sklearn.ensemble import RandomForestRegressor
reg_rf = RandomForestRegressor()
reg_rf.fit(X_train, y_train)
```

```
y_pred = reg_rf.predict(X_test)
```

```
reg_rf.score(X_train, y_train)
```

```
0.9532259326176381
```

```
reg_rf.score(X_test, y_test)
```

```
0.7989907296612544
```

```
sns.distplot(y_test-y_pred)
plt.show()
```

Plotting the Normal distribution/ Histogram graph for the Density and Price.

```
plt.figure(figsize=(10,8))
sns.distplot(df["Price"],bins=100,rug=True,color="g",label="Price")
plt.title("Distplot of Price",fontsize=15)
plt.legend()
```

```
plt.scatter(y_test, y_pred, alpha = 0.5)
plt.xlabel("y_test")
plt.ylabel("y_pred")
plt.show()
```

plotting the Histogram and Scatter plot to get the Visualisation.

```
from sklearn import metrics
print('MAE:', metrics.mean_absolute_error(y_test, y_pred))
print('MSE:', metrics.mean_squared_error(y_test, y_pred))
print('RMSE:', np.sqrt(metrics.mean_squared_error(y_test, y_pred)))
```

```
MAE: 1174.664996527429
MSE: 4334173.208722696
RMSE: 2081.8677212355965
```

Displaying the metrics like Mean Absolute Error, Mean Squared Error and Root Mean Squared Error.

MAE : 1174.664996527429

MSE : 4334173.208722696

RMSE : 2081.8677212355965

```
metrics.r2_score(y_test, y_pred)
```

```
0.7989907296612544
```

The accuracy of the Random Forest model after training is **0.7989907296612544**, that is **79%**.

4.11 Fitting Model With Hyperparameter Tuning

Hyperparameter tuning is the process of finding the optimal set of hyperparameters for a machine learning model. Hyperparameters are the configuration settings for a machine learning model that are set before training the model.

We'll use RandomizedCVSearch Method for hyperparameter tuning.

```
from sklearn.model_selection import RandomizedSearchCV
```

```
#Randomized Search CV

# Number of trees in random forest
n_estimators = [int(x) for x in np.linspace(start = 100, stop = 1200, num = 12)]
# Number of features to consider at every split
max_features = ['auto', 'sqrt']
# Maximum number of levels in tree
max_depth = [int(x) for x in np.linspace(5, 30, num = 6)]
# Minimum number of samples required to split a node
min_samples_split = [2, 5, 10, 15, 100]
# Minimum number of samples required at each leaf node
min_samples_leaf = [1, 2, 5, 10]
```

```
# Create the random grid

random_grid = {'n_estimators': n_estimators,
               'max_features': max_features,
               'max_depth': max_depth,
               'min_samples_split': min_samples_split,
               'min_samples_leaf': min_samples_leaf}
```

Creating the random grid for estimators, max_features, min_features, min_samples_split and min_samples_leaf.

```
rf_random = RandomizedSearchCV(estimator = reg_rf,
param_distributions = random_grid,scoring='neg_mean_squared_error',
n_iter = 10, cv = 5, verbose=2, random_state=42, n_jobs = 1)
```

Initiating the Hyperparameter Tuning model by the Randomized Search CV performer for 50 folds and 10 iterations.

```
rf_random.fit(X_train,y_train)
```

The code trains a random forest regression model with hyperparameter optimization using a randomized search over a specified hyperparameter grid, and cross-validation is used to evaluate the performance of the model. The **fit()** method is then called to perform the hyperparameter search and train the final model using the best hyperparameters found during the search.

```
Fitting 5 folds for each of 10 candidates, totalling 50 fits
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=5, min_samples_split=5, n_estimators=900; total time=   5.7s
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=5, min_samples_split=5, n_estimators=900; total time=   4.3s
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=5, min_samples_split=5, n_estimators=900; total time=   4.1s
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=5, min_samples_split=5, n_estimators=900; total time=   5.5s
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=5, min_samples_split=5, n_estimators=900; total time=   4.4s
[CV] END max_depth=15, max_features=sqrt, min_samples_leaf=2, min_samples_split=10, n_estimators=1100; total time=   6.7s
[CV] END max_depth=15, max_features=sqrt, min_samples_leaf=2, min_samples_split=10, n_estimators=1100; total time=   7.9s
[CV] END max_depth=15, max_features=sqrt, min_samples_leaf=2, min_samples_split=10, n_estimators=1100; total time=   6.7s
[CV] END max_depth=15, max_features=sqrt, min_samples_leaf=2, min_samples_split=10, n_estimators=1100; total time=   7.7s
[CV] END max_depth=15, max_features=sqrt, min_samples_leaf=2, min_samples_split=10, n_estimators=1100; total time=   6.8s
[CV] END max_depth=15, max_features=1.0, min_samples_leaf=5, min_samples_split=100, n_estimators=300; total time=   0.0s
[CV] END max_depth=15, max_features=1.0, min_samples_leaf=5, min_samples_split=100, n_estimators=300; total time=   0.0s
[CV] END max_depth=15, max_features=1.0, min_samples_leaf=5, min_samples_split=100, n_estimators=300; total time=   0.0s
[CV] END max_depth=15, max_features=1.0, min_samples_leaf=5, min_samples_split=100, n_estimators=300; total time=   0.0s
[CV] END max_depth=15, max_features=1.0, min_samples_leaf=5, min_samples_split=100, n_estimators=300; total time=   0.0s
[CV] END max_depth=15, max_features=1.0, min_samples_leaf=5, min_samples_split=5, n_estimators=400; total time=   0.0s
[CV] END max_depth=15, max_features=1.0, min_samples_leaf=5, min_samples_split=5, n_estimators=400; total time=   0.0s
[CV] END max_depth=15, max_features=1.0, min_samples_leaf=5, min_samples_split=5, n_estimators=400; total time=   0.0s
[CV] END max_depth=15, max_features=1.0, min_samples_leaf=5, min_samples_split=5, n_estimators=400; total time=   0.0s
[CV] END max_depth=15, max_features=1.0, min_samples_leaf=5, min_samples_split=5, n_estimators=400; total time=   0.0s
[CV] END max_depth=20, max_features=1.0, min_samples_leaf=10, min_samples_split=5, n_estimators=700; total time=   0.0s
[CV] END max_depth=20, max_features=1.0, min_samples_leaf=10, min_samples_split=5, n_estimators=700; total time=   0.0s
[CV] END max_depth=20, max_features=1.0, min_samples_leaf=10, min_samples_split=5, n_estimators=700; total time=   0.0s
[CV] END max_depth=20, max_features=1.0, min_samples_leaf=10, min_samples_split=5, n_estimators=700; total time=   0.0s
[CV] END max_depth=20, max_features=1.0, min_samples_leaf=10, min_samples_split=5, n_estimators=700; total time=   0.0s
[CV] END max_depth=25, max_features=sqrt, min_samples_leaf=1, min_samples_split=2, n_estimators=1000; total time=  11.3s
[CV] END max_depth=25, max_features=sqrt, min_samples_leaf=1, min_samples_split=2, n_estimators=1000; total time=  11.3s
[CV] END max_depth=25, max_features=sqrt, min_samples_leaf=1, min_samples_split=2, n_estimators=1000; total time=  11.5s
[CV] END max_depth=25, max_features=sqrt, min_samples_leaf=1, min_samples_split=2, n_estimators=1000; total time=  10.8s
[CV] END max_depth=25, max_features=sqrt, min_samples_leaf=1, min_samples_split=2, n_estimators=1000; total time=  10.2s
```

```
estimator=RandomForestRegressor(bootstrap=True,
                                ccp_alpha=0.0,
                                criterion='mse',
                                max_depth=None,
                                max_features='auto',
                                max_leaf_nodes=None,
                                max_samples=None,
                                min_impurity_decrease=0.0,
                                min_impurity_split=None,
                                min_samples_leaf=1,
                                min_samples_split=2,
                                min_weight_fraction_leaf=0.0,
                                n_estimators=100,
                                n_jobs=None, oob_score=False,
                                random_state=None, verbose=0,
                                warm_start=False),
iid='deprecated', n_iter=10, n_jobs=-1,
param_distributions={'max_depth': [5, 13, 21, 30],
                     'max_features': ['auto', 'sqrt'],
                     'min_samples_split': [5, 10, 15, 100],
                     'n_estimators': [100, 320, 540, 760,
                                      980, 1200]},
pre_dispatch='2*n_jobs', random_state=None, refit=True,
return_train_score=False, scoring=None, verbose=2)
```

This is the output for 50 folds and the description of data after model is trained.

```
rf_random.best_params_
```

```
{'n_estimators': 700,
 'min_samples_split': 15,
 'min_samples_leaf': 1,
 'max_features': 'auto',
 'max_depth': 20}
```

These are the best parameters that are yield from the model.

```
prediction=rf_random.predict(X_test)
sns.distplot(y_test-prediction)
metrics.r2_score(y_test,prediction)
```

```
0.8581264566050444
```

After the Model is trained, the Accuracy is as shown.

```
print('MAE',metrics.mean_absolute_error(y_test,prediction))
print('MSE',metrics.mean_squared_error(y_test,prediction))
print('RMSE',np.sqrt(metrics.mean_squared_error(y_test,prediction)))
```

```
MAE 1066.0691051282292
MSE 2590383.3756028726
RMSE 1609.4667985400856
```

Displaying the metrics like Mean Absolute Error, Mean Squared Error and Root Mean Squared Error.

MAE : 1066.0691051282292

MSE : 2590383.3756028726

RMSE : 1609.4667985400856

4.12 Saving The Model For Pickle

Pickle is a Python module that allows you to serialize and deserialize Python objects. You can use pickle to save a trained model for flight fare prediction so that you can reuse it later without having to retrain it every time you want to make a prediction.

```
import pickle
# open a file, where you ant to store the data
file = open('flight_rf.pkl', 'wb')

# dump information to that file
pickle.dump(reg_rf, file)
```

```
model = open('flight_rf.pkl','rb')
forest = pickle.load(model)
```

```
y_prediction = forest.predict(X_test)
```

The model is further used to develop the Web application, so pickled the data and model into FlightFare.pkl file to use it in Python with Flask and Pickle libraries.

4.13 Final Accuracy Of The Model

```
metrics.r2_score(y_test,prediction)

0.8581264566050444
```

The final Accuracy of the model after performing Random Forest and Hyperparameter Tuning is **0.8581264566050444** that is **85.81%.**

4.14 Features:

Airline: The name of the airline.

Date_of_Journey: The date of the journey

Source: The source from which the service begins.

Destination: The destination where the service ends.

Route: The route taken by the flight to reach the destination.

Dep_Time: The time when the journey starts from the source.

Arrival_Time: Time of arrival at the destination.

Duration: Total duration of the flight.

Total_Stops: Total stops between the source and destination.

Additional_Info: Additional information about the flight

Price: The price of the ticket A regression problem in which duration, source, destination, dates of flight and route are provided.

CHAPTER 5

TESTING

Testing is an integral part of developing any operational system. Testing is a method to check whether the actual product matches expected requirements and to ensure that product is Defect free. It involves the execution of system components using manual or automated tools to evaluate one or more properties of interest. The purpose of testing is to identify errors, gaps or missing requirements in contrast to actual requirements.

5.1 Model Testing

Model testing is a crucial step in developing and maintaining an airfare prediction system. Here are some best practices to follow when testing the model in such a system:

1. Define a testing strategy: Determine what types of tests you need to run, such as unit tests, integration tests, or end-to-end tests. Identify the performance metrics you will use to measure the model's accuracy and reliability, such as mean absolute error or root mean squared error.

2. Collect and prepare test data: Gather a diverse set of test data that includes a range of different flight routes, travel dates, and other variables that may affect airfare prices. Preprocess this data as you would in your production system, and ensure that it is representative of the data the model will encounter during normal operation.

3. Train and validate the model: Train the model on a subset of the data and use the remainder for validation. Use techniques such as cross-validation to ensure that the model is not overfitting to the training data. Evaluate the model's performance on the validation set using the metrics you defined in step 1.

4. Test the model: Use the test dataset you collected which is used to evaluate the model's performance on new and unseen data. Compare the model's predictions to the actual airfare prices and calculate the performance metrics you defined in step 1.

5. Analyze and refine: Analyze the model's performance and identify areas where it may be lacking. Refine the model by adjusting hyperparameters, changing the model architecture, or adding new features to the input data. Repeat steps 3-5 until you are satisfied with the model's performance.

5.2 Unit Testing

Unit testing is a type of software testing where individual units or components of a software are tested. The purpose is to validate that each unit of the software code performs as expected. Unit Testing is done during the development (coding phase) of an application by the developers. Unit Tests isolate a section of code and verify its correctness. A unit may be an individual function, method, procedure, module, or object.

Unit Testing of the Airfare Prediction System contains

Identify the components that need to be tested: In a flight price prediction system, there may be multiple components that need to be tested, such as data pre-processing, model training, and price prediction algorithms. Identify these components and create a testing plan for each.

Write test cases: For each component, write test cases that cover all the different scenarios that the component may encounter during normal operation. For example, you may want to test the model's performance on various types of flight data, such as domestic versus international flights, short-haul versus long-haul flights, and peak versus off-peak travel periods.

Automate tests: Automating tests is essential in any software development project. You can use tools such as pytest, unit test, or Selenium to automate your tests and run them regularly.

Mock dependencies: In a flight price prediction system, you may have dependencies on external services or data sources, such as weather data or airline schedules. To isolate the component being tested, you can use mocking frameworks to simulate the behavior of these dependencies.

Measure coverage: To ensure that your tests are comprehensive, measure code coverage regularly. This will help you identify areas of the code that are not being tested and ensure that your tests are effective in catching bugs and errors.

Run tests in a continuous integration (CI) pipeline: Incorporating your unit tests into a CI pipeline will ensure that they are run automatically each time code is committed to the codebase. This will help catch issues early in the development process and ensure that your code is always stable and reliable.

5.3 Performance Testing

Performance testing checks the speed, response time, reliability, resource usage, and scalability of a software program under its expected workload. The purpose of Performance Testing is not to find functional defects but to eliminate performance bottlenecks in the software or device. The focus of Performance Testing is checking a software program's

- Speed - The Time that the system takes to generate the output by training the model.
- Scalability - The maximum load of how many input streams can the system handle at a time
- Stability -The ability of the system to stay stable, even in the face of unpredictable situations and unpredictable loads.

CHAPTER 6
RESULTS AND DISCUSSION

The system has various steps that produce multiple outputs all these are needed at different stages in the prediction process. There are mainly three stages in the whole system they include

- Data Handling.
- Model Training.
- Model Deployment.

During these stages we will be having multiple outputs. They include:

6.1 Data Handling:

This includes organizing the unordered data in our Dataset. There is a lot of redundant data present in the dataset, so in order to remove that we follow some steps. This consists of two stages:

- One Hot Encoding.
- Label Encoding.
- Handling Categorical Data

6.1.1 One Hot Encoding:

One hot encoding is a technique used to represent categorical variables as numerical values in a machine learning model. The advantages of using one hot encoding include:

- It allows the use of categorical variables in models that require numerical input.
- It can improve model performance by providing more information to the model about the categorical variable.
- It can help to avoid the problem of ordinality, which can occur when a categorical variable has a natural ordering (e.g. “small”, “medium”, “large”).

After performing One hot encoding the output is shown in the Figure 6.1. We can see that it is in the form of a table having indices and labels with respect to the features in the matrix.

```
print(pd.DataFrame(X_train).head())
```

```
   0  1  2    3  4  5   6  7   8   9   10  11  12  13
0  3  0  5   18  4  8  24  3   2  50  22  20   1  10
1  1  3  0   84  1  8   1  5   7  25   5  50  13  15
2  4  2  1  118  1  8   9  6  19   0   9  25   4  25
3  3  3  0   91  0  8  12  5   5  25  18   5  23  30
4  3  0  5   29  0  8   1  3   4  45  16  50  21  35
```

Figure 6.1 Output of One Hot Encoding

6.1.2 Label Encoding:

Label Encoding refers to converting the labels into a numeric form so as to convert them into the machine-readable form. Machine learning algorithms can then decide in a better way how those labels must be operated. It is an important pre-processing step for the structured dataset in supervised learning.

```
#Label encoding
train_data.replace({"non-stop": 0, "1 stop": 1, "2 stops": 2, "3 stops": 3, "4 stops": 4}, inplace = True)
```

```
train_data.head()
```

	Airline	Source	Destination	Total_Stops	Price	Journey_day	Journey_month	Dep_hour	Dep_min	Arrival_hour	Arrival_min	Duration_hours	Duration_mins
0	IndiGo	Banglore	New Delhi	0	3897	24	3	22	20	1	10	2	50
1	Air India	Kolkata	Banglore	2	7662	1	5	5	50	13	15	7	25
2	Jet Airways	Delhi	Cochin	2	13882	9	6	9	25	4	25	19	0
3	IndiGo	Kolkata	Banglore	1	6218	12	5	18	5	23	30	5	25
4	IndiGo	Banglore	New Delhi	1	13302	1	3	16	50	21	35	4	45

Figure 6.2 Output of Label Encoding

6.1.3 Categorical Data:

Categorical data is a set of predefined categories or groups an observation can fall into. Categorical data can be found everywhere. For instance, survey responses like marital status, profession, educational qualifications, etc. However, certain problems can arise with categorical data that must be dealt with before proceeding with any other task.

So, let us take a look at some problems posed by categorical data and how to handle them. After performing this other steps as mentioned above are performed. This includes Cleaning, Encoding, Labelling etc.

In the Figure 6.3 we have the output of average price of each Airline.

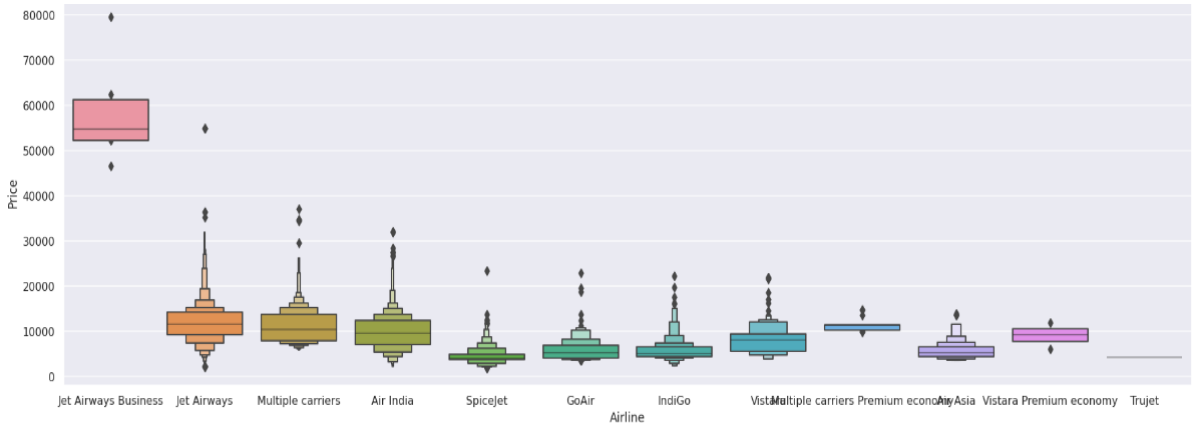


Figure 6.3 Airline vs Price

In the Figure 6.4 we have the output of price variation due to destination.

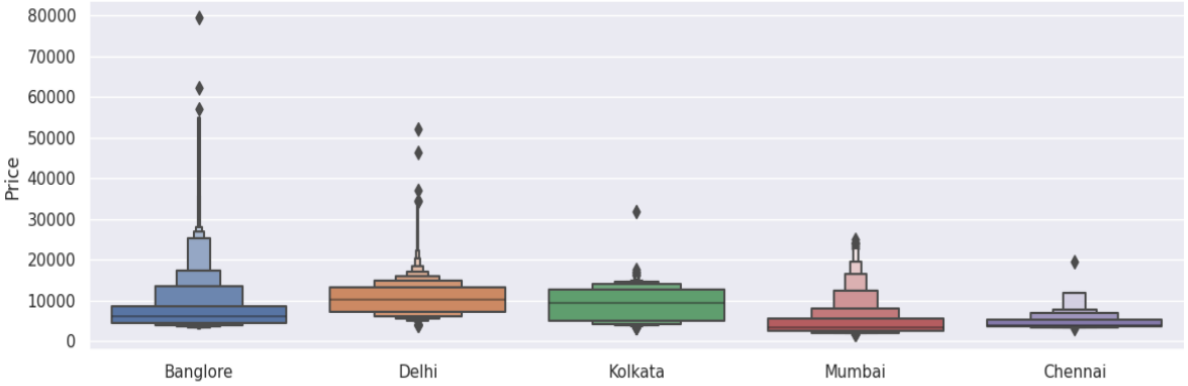


Figure 6.4 Destination vs price

6.2 Model Training:

After performing Data Handling part, we are provided with the target features as output. This step is achieved in the Feature Extraction stage. Feature extraction is the process of transforming raw data into a set of features that can be used to represent the data in a machine learning model. The goal of feature extraction is to create a set of features that captures the most relevant information from the raw data and is informative for the machine learning model.

After this the extracted features are used as parameters for performing the operations on data. These are used as references for the algorithm. Figure 6.5 shows the output data of Y-Feature.

```
#Dependent/Target variable => Output
y = data_train.iloc[:, 1]
y.head()
```

```
0     3897
1     7662
2    13882
3     6218
4    13302
Name: Price, dtype: int64
```

Figure 6.5 Output by Independent Feature

6.2.1 Fitting Model with Random Forest Algorithm:

Now we apply the algorithm by introducing the target variables as parameters. Upon completing that the output is shown in Figure below.

```
reg_rf.score(X_test, y_test)
```

```
0.7989907296612544
```

```
sns.distplot(y_test-y_pred)
plt.show()
```

Plotting the Normal distribution/ Histogram graph for the Density and Price in Figure 6.6.

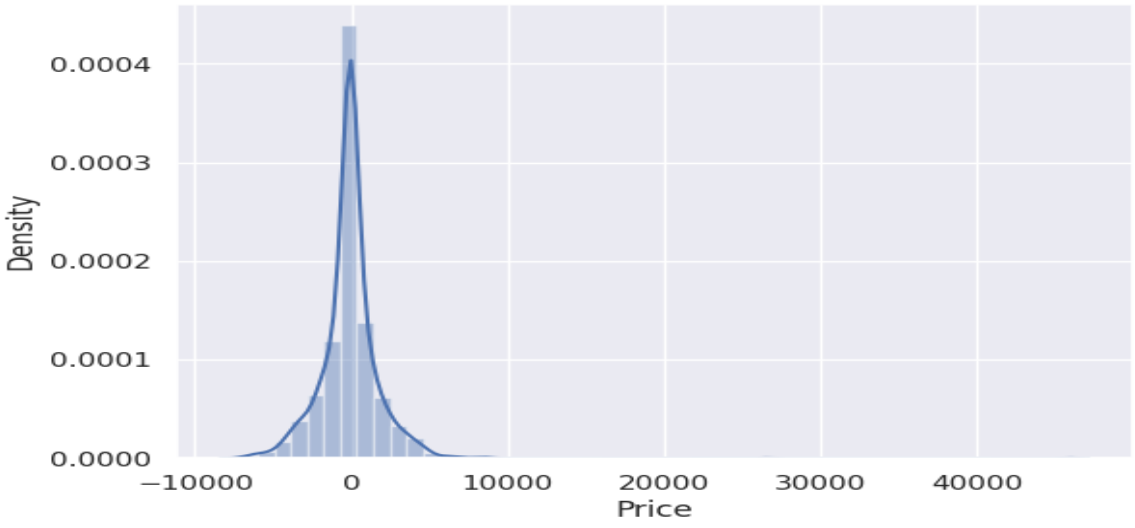


Figure 6.6 Random Forest Distribution

6.2.2 Hyper Parameter Tuning:

Hyper parameter tuning is the process of finding the optimal set of hyperparameters for a machine learning model. Hyperparameters are the configuration settings for a machine learning model that are set before training the model.

Before Hyper Parameter Tuning:

The Figure 6.7 and Figure 6.8 describes Accuracy and Output before Tuning. In Figure 6.7 we can see the accuracy of model to be approximately 35%. In the Figure 6.8 we can see the density of the outcomes. To be clear high density corresponds to high efficiency.

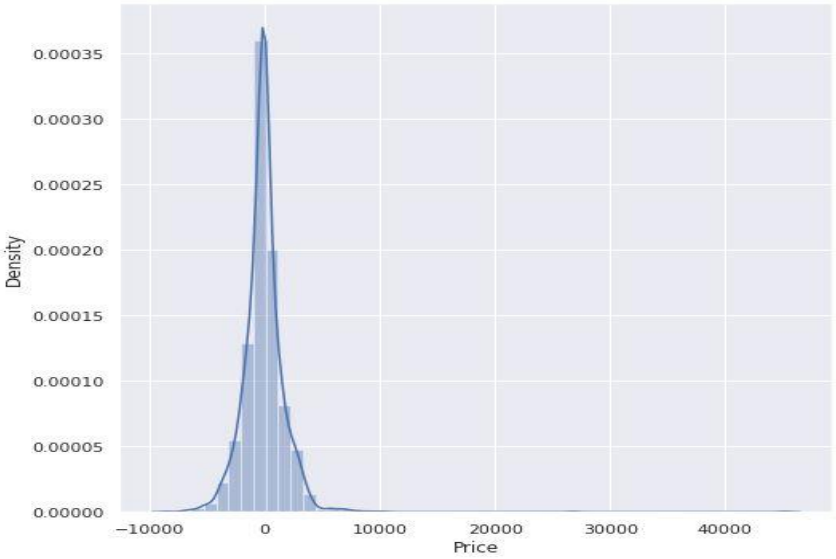


Figure 6.7 Accuracy of trained model before Tuning.

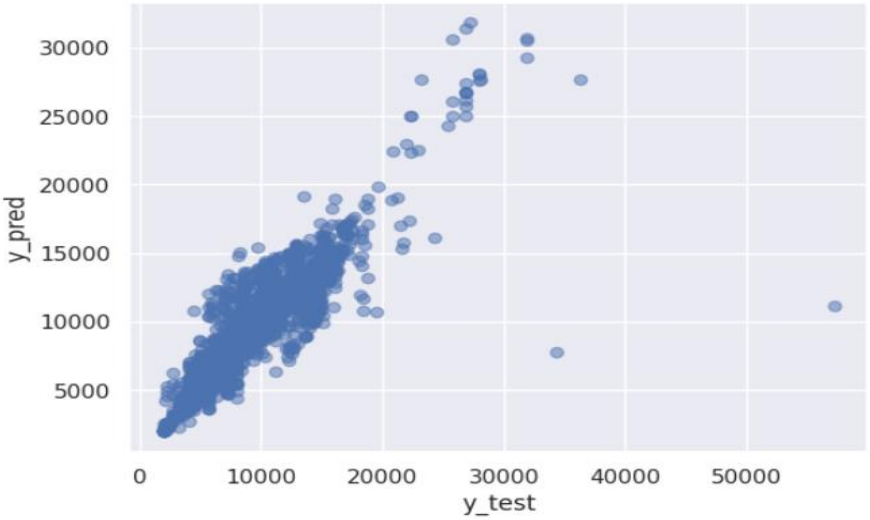


Figure 6.8 Output before Tuning.

After Hyper Parameter Tuning:

The Figure 6.9 and Figure 6.10 describes Accuracy and Output after Tuning. In Figure 6.9 we cans see that, since the parameter tuning process has been performed the accuracy has risen to 40% and this indicates that the accuracy of model has been improved.

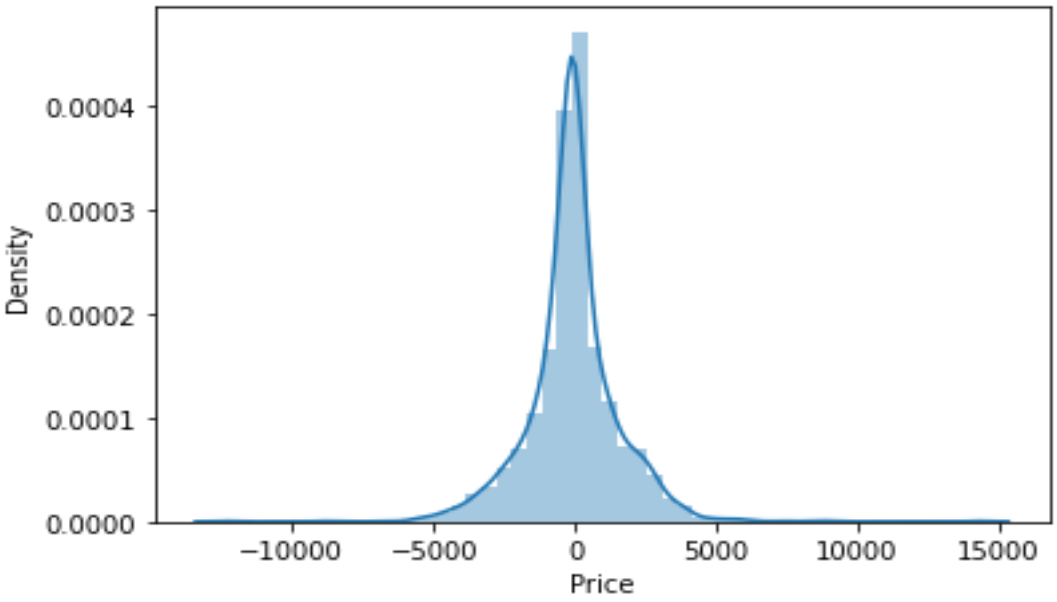


Figure 6.9 Accuracy after Tuning.

In the Figure 6.10 we can see the density of the predicted outcomes is very high. This shows that the model has improved efficiency.

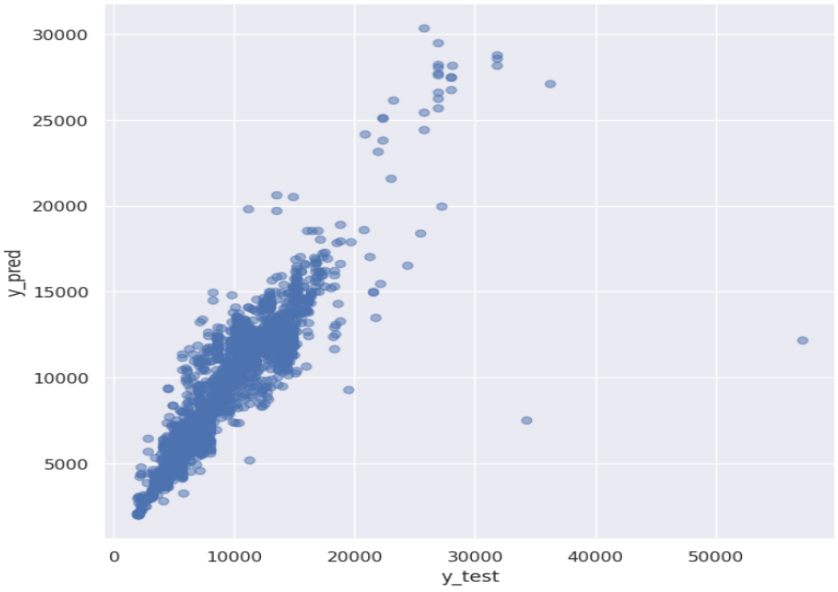


Figure 6.10 Output after Tuning.

6.3 Correlation for Independent and Dependent Features:

In this process, we will find out the best feature which will contribute to our target variable.

X = “Independent Feature”.

Y = “Dependent Feature” i.e., “Price” column.

We will separate all the independent features except price in the X variable and price in Y variable. The Figure 6.11 shows the correlation between X and Y in the form of a Heat Map.

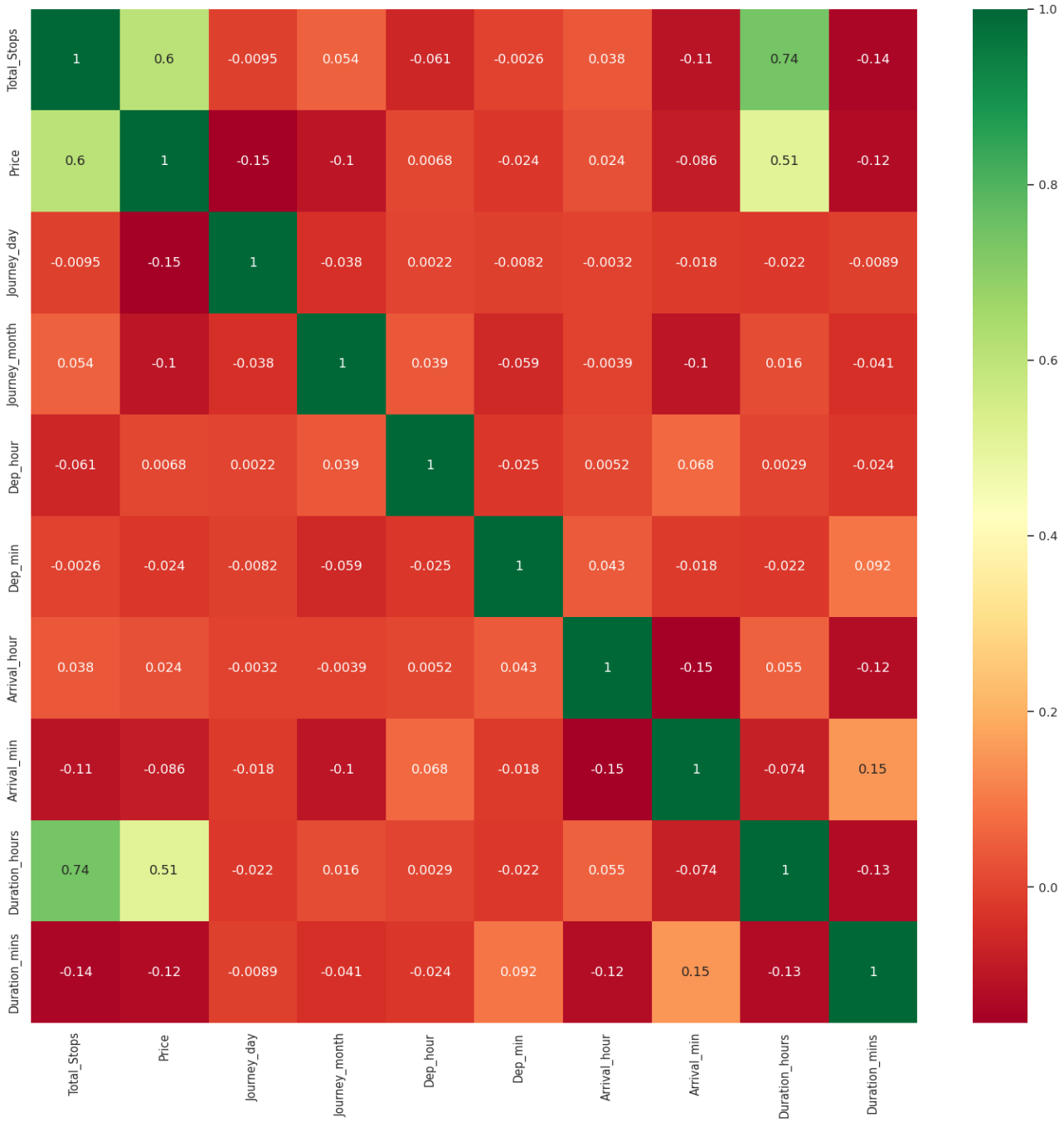


Figure 6.11 Heat Map for X and Y Features

6.4 Feature Selection:

We have used “ExtraTreesRegressor” to find more important features from the data. Use the election variable and do fitting the X & Y features. Figure 6.12 shows about the Feature Selection. After this we will print “feature_importance” and will get to know the important features.

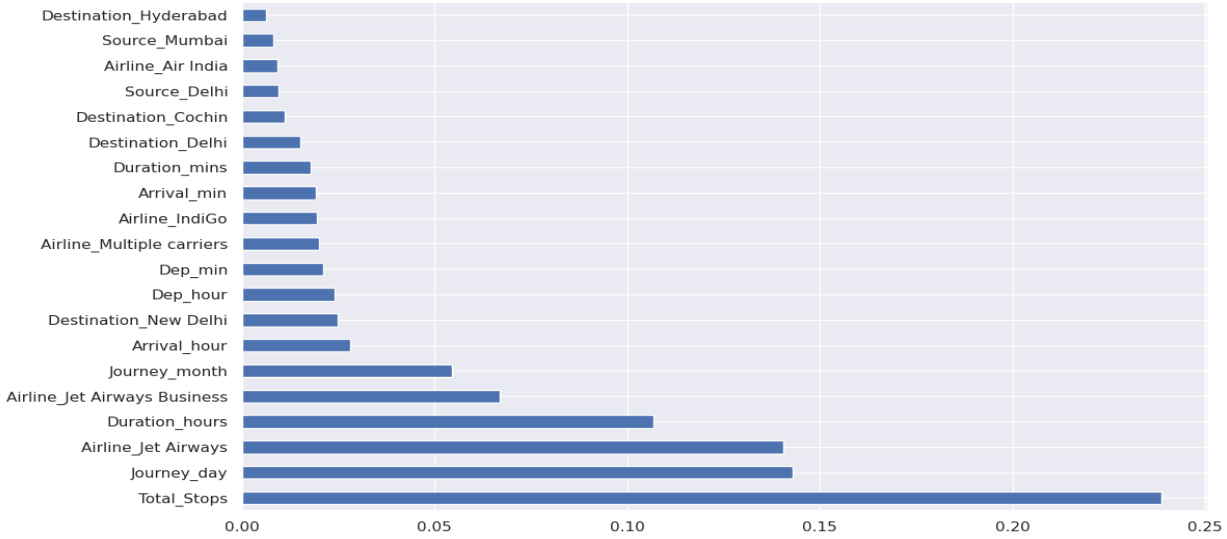


Figure 6.12 Feature Selection

6.5 Model Deployment:

After successfully training the model we deploy it with the help of flask. Flask is a Python-based micro framework used for developing small-scale websites. Flask is very easy to make Restful APIs using python. As of now, we have developed a model i.e. model.pkl , which can predict a class of the data based on various attributes of the data as shown in Figure 6.13.

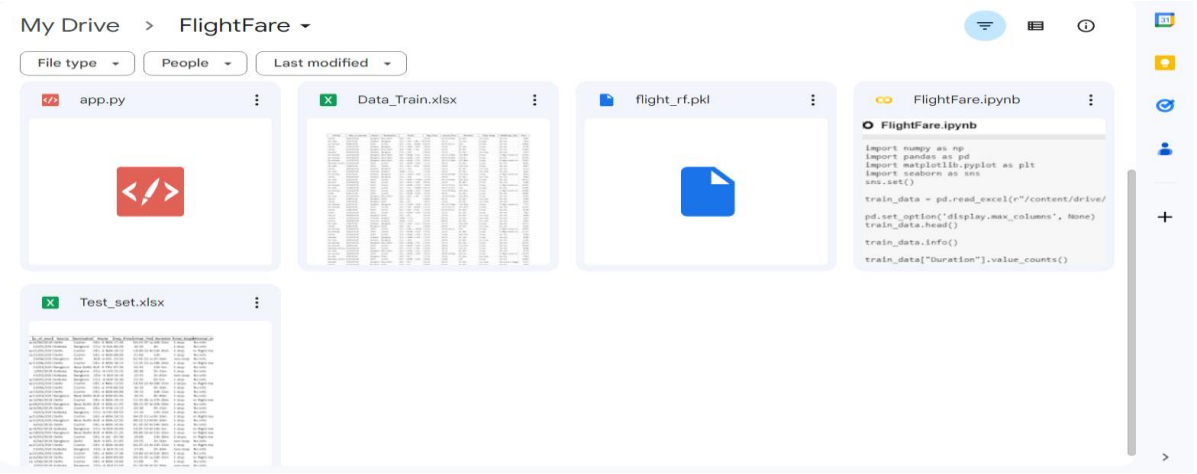


Figure 6.13 Deployment of (Flight_rl.pkl) pickle file in root directory of Google Drive

CHAPTER 7
CONCLUSION AND FUTURE WORK

This project used deep learning techniques, specifically the Random Forest algorithm, to predict airfare prices. The system consisted of three stages: Feature Extraction, Hyperparameter Tuning, and Feature Extraction. The trained model achieved an accuracy of 85% using Python libraries. Limitations include assumptions about system capability, potential inaccuracies, and limitations to domestic airlines. Future work could involve real-time data, addressing missing data and external factors, expanding system features, and improving user interface for a more robust Flight Fare Prediction system.

7.1 Conclusion

This project has investigated the use of deep learning techniques in the field of Airfare Prediction systems. The proposed system consists mainly of three stages. They are Feature Extraction, Hyper parameter Tuning and Feature Extraction. This project uses a trained price prediction model which helps to improve the accuracy of price prediction. This technique helps in the extraction of features that affect the price of a flight ticket.

Another important aspect incorporated in this project is using Random forest algorithm for merging multiple models into one unified model. The prediction task includes the prediction of ticket prices. Our study also includes how the data format impacts the prediction system and different techniques to optimize Random forest algorithm.

Our experiments show an overall accuracy of 85% in predicting the prices successfully. Our Algorithm is implemented using NumPy, and pandas libraries in Python environments. In general training duration for our Algorithm is 2-4 Hours, which depends on the neural network architecture. This system can be used for practical use by the following improvements

1. Usage of real-time data to train the Random forest algorithm.
2. Missing data values.
3. External factors like climatic conditions.

Table 7.1 Test and Train Results

ML Algorithm	Train Data	Test Data
Random Forest	0.7774	0.7752
Hyper Parameter Tuning	0.9529	0.7970

Table 7.1shows test results for the train data and test data run over different algorithms.

Table 7.2 Accuracy Metric Results

ML Algorithm	MAE	MSE	RMSE
Random Forest	1107.446856173	3106839.70605	1762.622961967
Hyper Parameter Tuning	1066.069105128	2590383.375602	1609.466798540

These are the accuracy metrics that are derived from the algorithms.

From the Table 7.2,

- MAE : Mean Absolute Error
- MSE : Mean Squared Error
- RMSE : Root Mean Squared Error

Table 7.3 ML Model Accuracy

ML Model	Accuracy	Percentage (%)
Random Forest	0.7989907296612544	79.8%
Hyper Parameter Tuning	0.8581264566050444	85.8%

According to the Table 7.3, As we can see Hyper Parameter Tuning is performing better than Random Forest algorithm. So, we have chosen Hyper Parameter Tuning Algorithm for the project which yielded **85.8%** Accuracy.

7.2 Limitations

The system shows satisfactory results in the segmentation stage and recognition stage. There are certain limitations to the system as it is developed with certain assumptions. The limitations are as follows:

1. It is assumed that the system is having enough capability to run the Algorithm.
2. As the accuracy is only 85% in some worst cases it gives the wrong price as output.
3. It can only predict the prices of domestic airlines.
4. Ticket booking option is not available here.

7.3 Future Work

The Flight Fare Prediction system developed in this project has a lot of capabilities apart from mere price prediction. We will include notifying the customers about the airport facilities and reviews of the particular airport. Future work includes ticket booking feature built in with this software. Apart from all these the UI and other functional elements of the software will be improved for easier usage by all age groups.

REFERENCES

1. Tianyi wang, samira Pouyanfar, haiman Tian and Yudong Tao "A Framework for airline price prediction: A machine learning approach"

2. Yekta Said Can; Koray Büyükoğuz; Efe Batur Giritli; Mustafa Şişik; Fatih Alagöz ,Predicting Airfare Price Using Machine Learning Techniques: A Case Study for Turkish Touristic Cities, Publisher: IEEE,2022

3. R. Raja Subramanian; Marisetty Sai Murali; B Deepak; P Deepak; Hamsinipally Nikhil Reddy; R. Raja Sudharshan Airline Fare Prediction Using Machine Learning Algorithms ,Publisher: IEEE,2022

4. William Groves and Maria Gini "An agent for optimizing airline ticket purchasing" in proceedings of the 2013 international conference on autonomous agents and multiagent systems.

5. https://www.javatpoint.com/machine-learningrandom-forest-algorithm Random Forest Algorithm

6. https://www.analyticsvidhya.com/blog/2021/04/rapid-fire-eda-process-using-python-for-ml implementation/. Rapid-Fire EDA process using Python for ML Implementation

7. https://www.irjet.net/archives/V9/i5/IRJET-V9I5227.pdf